Algoritmi per l'intelligenza artificiale

Giuseppe Di Blasi
Università di Parma
giuseppe.diblasi@studenti.unipr.it

A.A. 2026/2027

ABSTRACT

Appunti del corso di Algoritmi per l'intelligenza artificiale.

Indice

Indice

1 Lezione 1 del 21 settembre 2026 3
1.1 Introduzione del corso 3
1.2 Apprendimento automatico 3
1.3 Apprendimento Semi-supervisionato 5
1.4 Apprendimento non supervisionato 5
2 Lezione 2 del 25 settembre 2026 8
2.1 Classificazione 8
2.2 Classificazione Iris di Fisher 10
2.3 Algoritmi di apprendimento 10
2.4 Perché l'informatica è una scienza? 13
3 Lezione 3 del 28 settembre 2026 14
4 Lezione 4 del 2 ottobre 2026 17
4.1 Indici statistici delle distribuzioni 17
4.2 Confrontare distribuzioni 21
5 Lezione 5 del 5 ottobre 2026 23
5.1 Classificazione: tecniche di validazione 25
5.2 Cross-validation 30
6 Lezione 6 del 9 ottobre 2026 32
6.1 Algoritmi di classificazione 32
7 Lezione 7 del 16 ottobre 2026 45
7.1 Indice di Gini 45
7.2 Pruning degli alberi di decisione 47
7.3 Minimal cost-complexity pruning 48
7.4 Metodi di consenso 52
7.5 Boosting 57
8 Lezione 8 del 19 ottobre 2026 59
8.1 Support Vector Machine 59
8.2 SVM per dati non separabili 62
8.3 SVM non lineari 63
8.4 SVM: Estensione multiclasse 64

1 Lezione 1 del 21 settembre 2026

1.1 Introduzione del corso

L'obiettivo del corso è fornire basi teoriche per modellare e risolvere problemi di intelligenza artificiale tramite tecniche di apprendimento automatico e della ricerca di soluzioni da un punto di vista algoritmico.

Un approccio critico sull'utilizzo e comprensione degli algoritmi di intelligenza artificiale è tra gli obiettivi principali del corso, sopratutto identificare la corretta metodologia e valutarne l'impatto sul caso di applicazione specifico.

Detto dal prof, nel campo dell'informatica molti algoritmi e strutture dati prendono spunto dalla natura, per esempio le reti neurali cercano di rappresentare il più possibile la struttura di un cervello umano.

1.2 Apprendimento automatico

Per apprendimento automatico, o machine learning, si intende la capacità di una macchina di poter imparare qualcosa da un insieme di dati di partenza (di solito un dataset) per poter riconoscere dati simili che non ha mai visto durante la fase di training.

Ci sono 3 tipi di apprendimento automatico:
- Supervisionato
- Semi-supervisionato
- Non supervisionato
- Apprendimento per rinforzo (non verrà trattato a lezione)

La differenza principale tra queste 3 metodologie è la disponibilità dei dati di verità di base, o **ground truth**, ovvero prove empiriche che sono conoscenze preliminari di ciò che l'output del modello dovrebbe generare in base a un determinato input.

L'apprendimento automatico viene usato nei seguenti contesti:
- Classificazione: Quando vogliamo mappare l'input alle etichette di output
- Regressione: Quando vogliamo mappare l'input a un output continuo

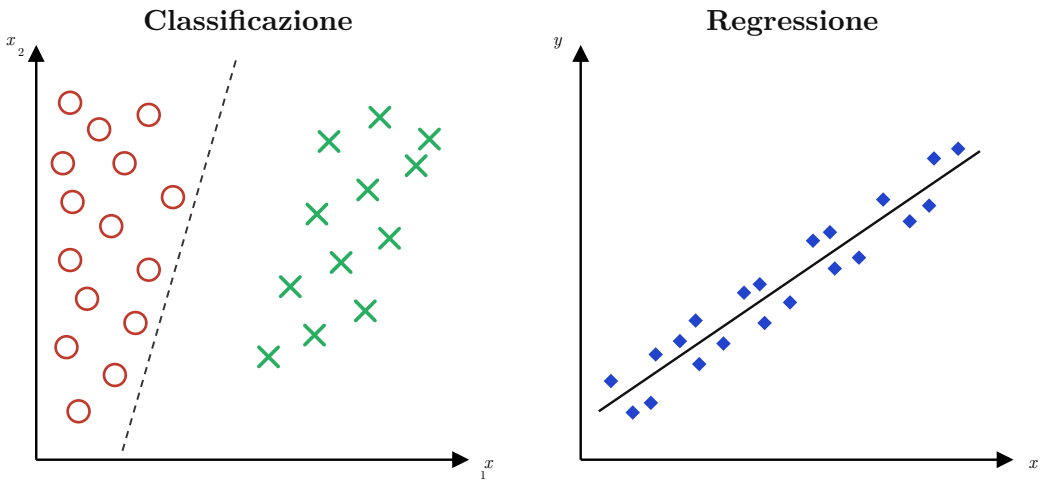


Figura 1. Classificazione vs Regressione: nella classificazione l'output è una classe discreta, nella regressione un valore continuo.

Nel caso della classificazione abbiamo classi linearmente separabili. Il problema più importante che accomuna i due contesti è capire come disegnare al meglio la retta, o la curva, per poter approssimare al meglio i dati che ci interessano.

Nella classificazione vogliamo separare le classi di dati, nella regressione vogliamo una retta che si avvicini al più possibile a tutti i dati che ci interessano. Dobbiamo prestare attenzione all'overfitting, ovvero evitare modelli troppo precisi perché ciò comporterebbe la perdità della capacità di poter riconoscere outlier, ovvero dati che appaiono di rado. Si dice che in questo caso la rete non **generalizzi**, ma impari a riprodurre i dati di training senza apprendere la tendenza o la struttura effettiva nei dati che porta all'output.

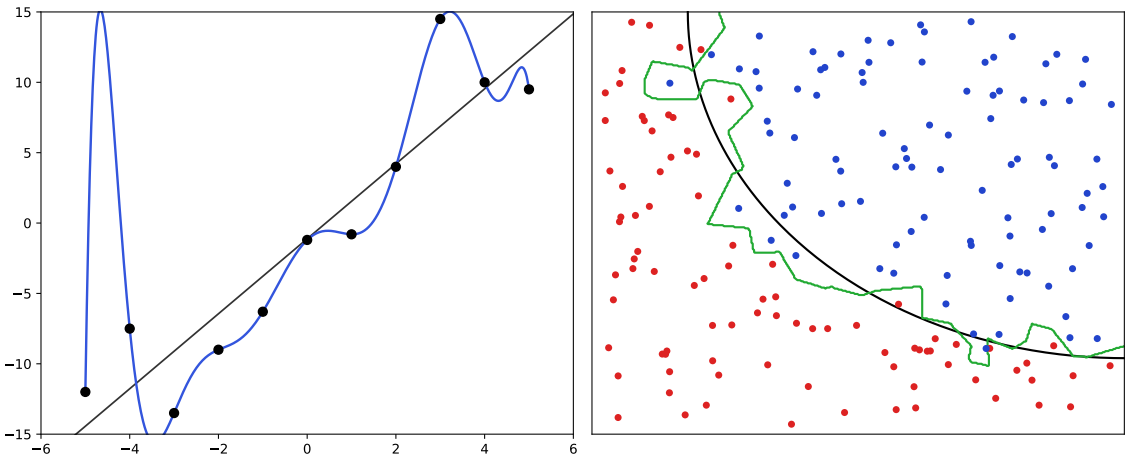


Figura 2. Overfitting in regressione (sinistra) e in classificazione (destra): il modello si adatta al rumore invece che al pattern sottostante.

Per poter approssimare i punti possiamo scegliere una qualunque curva aggiungendo quanta complessità vogliamo. Nel caso del grafico a sinistra, vediamo la parabola e la curva più complessa in colore verde. In questo caso ci conviene scegliere la parabola anche se commette errori per la classificazione, questo ci permette di evitare overfitting.

Per la generalizzazione del modello si parla anche di **compromesso bias-varianza**, ovvero cercare un equilibrio tra la distorsione (bias) e la varianza:

- Distorsione (bias): termine di errore costante
- Varianza: Quantità in base alla quale l'errore può variare tra diversi set di dati

Il problema è quello di creare un modello che riesca a catturare la regolarità dei dati di addestramento, ma che allo stesso tempo riesca a genarlizzare sui dati non usati nella fase di training.

Questo verrà visto più avanti.

1.3 Apprendimento Semi-supervisionato

Con l'apprendiemnto Semi-supervisionato si parla di metodi per poter lavorare su grandi quantità di dati dato che cercare di classificare ogni singola informazione sarebbe troppo dispendioso. Alcune tecniche che si utilizzano sono macchine vettoriali di supporto trasversali e metodi basati su grafi come la propagazione delle etichette. Per esempio se vogliamo filtrare determinati commenti su un social network, non possiamo etichettare tutti i commenti presenti sul sito, ma possiamo lavorare su un sottoinsieme etichettato.

I metodi semi-supervisionati devono fare alcune ipotesi sui dati al fine di giustificare l'utilizzo di una piccola serie di dati etichettati per trarre conclusioni sui punti non etichettati:

- Continuità: si presume che i punti vicini tra loro abbiano maggiori probabilità di avere un'etichetta in comune
- Ipotesi del cluster: si presume che i dati formino naturalmente cluster discreti e che i punti dello stesso cluster abbiano maggiori probabilità di condividere un'etichetta
- Presupposto molteplice: si presume che i dati si trovino approssimativamente in uno spazio di dimensioni inferiori (collettore) rispetto allo spazio di input. È uno scenario rilevante quando un sistema non osservabile o difficile da osservare con un numero ridotto di parametri produce output osservabili ad alta dimensione.

1.4 Apprendimento non supervisionato

L'apprendimento non supervisionato è utile per trovare modelli intrinseci nei dati che vogliamo studiare. Per esempio l'attività più comune per l'apprendimento non supervisionato è il clustering (raggruppamento), l'apprendimento della rappresentazione e la stima della densità.

Questo ci permette di conoscere la struttura intrinseca dei nostri dati senza utilizzare etichette. Alcuni algoritmi includono il clustering, l'analisi dei componenti principali e gli autocodificatori (autoencoders).

Abbiamo due tecniche per affrontare problemi di apprendimento non supervisionato:

- clustering
- riduzione della dimensionalità dei dati

1.4.1 Clustering

Il clustering è una tecnica esplorativa che consente di aggregare all'interno di gruppi, detti cluster, dei dati su cui non abbiamo una conoscenza preliminare di appartenenza a gruppi.

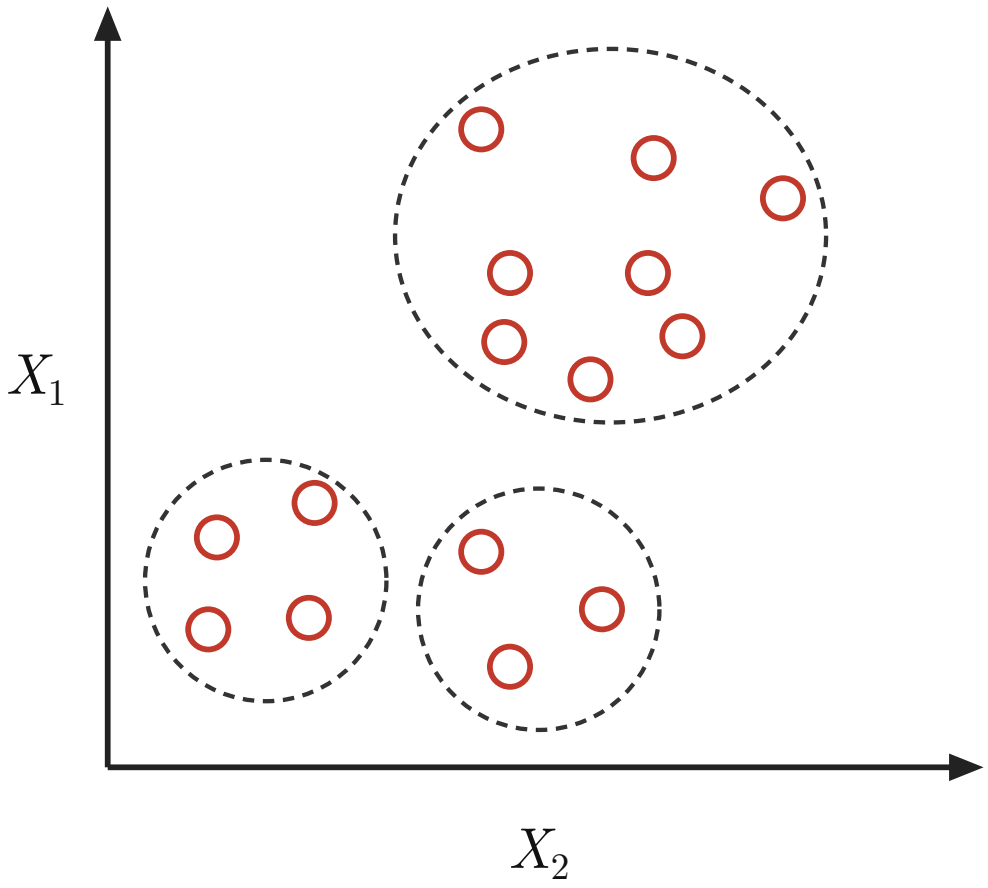


Figura 3. Rappresentazione dei cluster nel piano delle feature X_1 e X_2

Il clustering ci torna molto utile quando vogliamo trovare relazioni tra i dati di cui non abbiamo conoscenze pregresse.

1.4.2 Riduzione della dimensionalità

La riduzione della dimensionalità senza supervisione è un approccio molto utilizzato nella pre-elaborazione delle features (caratteristiche), con l'obiettivo di eliminare il rumore dai dati.

Questo può ridurre le prestazioni ma permette di rendere lo spazio dimensionale più compatto mantenendo le informazioni più rilevanti.

Questa tecnica è usata anche per dati ad elevata dimensionalità che possono essere proiettati su uno spazio 1D, 2D o 3D

1.4.3 Analisi esplorativa

L'apprendimento senza supervisione è utile per analisi esplorative perché permette di identificare automaticamente strutture all'interno dei dati.

In situazioni in cui è impossibile o impraticabile per un essere umano proporre tendeze nei dati, dovuta all'enorme mole su cui si lavora, l'apprendimento non supervisionato può fornire informazioni iniziali che possono essere utilizzate per testare o verificare singole ipotesi.

2 Lezione 2 del 25 settembre 2026

2.1 Classificazione

In questa lezione inzieremo a parlare di **classificazione**. L'obiettivo della **classificazione** è quello di dividere oggetti, osservando le loro caratteristiche, nelle loro classi di appartenenza.

Un esempio di classificazione è dividere le palle da calcio da quelle di basket, riconosciamo delle caratteristiche (**feature**) in ognuno degli oggetti che ci permette di dire se un determinato oggetto appartiene a una determinata classe oppure a un'altra.

> **Definizione 1 — Feature (Caratteristica)**
>
> Una **feature** è un aspetto **direttamente osservabile** relativo ad un fenomeno per il quale si può registrare uan precisa **misura quantitativa** (valore numerico intero o double) o **categoriale** (vero/falso, dolce/salato, festivo/feriale, rosso/verde/blu etcetera.)
>
> Le feature **categoriali** possono essere divise in:
> - **Ordinali**: possono essere confrontate direttamente
> - **Non-ordinali**: Non possono essere confrontate direttamente; per confrontarle si sfrutta **one-hot encoding**, dobbiamo quindi trovare un modo per rappresentarle e calcolare la loro distanza.

Il caso con 2 classi è quello più semplice, che prende il nome di **classificazione binaria**, nella realtà lavoreremo su problemi di classificazione con più di 2 classi. Modelli che riconosco oggetti tra 2 classi sono detti **modelli semplici**.

Ovviamente non classificheremo gli oggetti manualmente, ma sfrutteremo dei modelli automatizzati per poter risolvere questo problema. Diventa quindi fondamentale la scelta di un determinato modello di classificazione e anche come misurare le sue prestazioni. Prestazioni e problema da risolvere vanno di pari passo, se dovessimo scegliere un modello poco performante potremmo perdere efficienza nella classificazione. Anche le metriche o feature, che vogliamo minimizzare o massimizzare influiscono sulla scelta del modello più opportuno.

Il nostro modello, che può essere chiamato anche **classificatore**, partendo dall'osservazione di varie feature giunge alla decisione di **etichettare** (in inglese: **to label**) il dato osservato in una categoria più astratta e generale.

La classe di appartenenza è qualcosa che di solito diamo per scontato, in realtà è molto soggettiva, siamo noi a decidere quali feature riteniamo importanti per poter separare gli oggetti in determinate classi.

> **Esempio 1 — Classificazione uomini e donne**
>
> Ipotizziamo di voler risolvere il problema di classificazione dovendo distinguere esseri umani tra sesso maschile e femminile.
>
> Possiamo identificare varie feature che ci permettono di distinguere gli uni dagli altri:

- Lineamenti del volto
- Proporzioni corporee
- Vestiti
- Timbro di voce e così via

Dobbiamo tenere in considerazione che le *misurazioni* che vengono effettuate sulle feature da noi scelte, sono misurazioni *indirette*. Le feature che misuriamo sono in realtà dovute al DNA di una persona, quindi quello che osserviamo è una causa nella struttura del DNA. Per esempio potremmo avere degli **outlier** statistici, che vedremo più avanti nel corso, dove possiamo avere uomini che assomigliano a donne o viceversa, ma la possibilità di decidere oggettivamente è contenuta all'interno DNA.

Nella realtà è più una zona grigia la distinzione netta tra due classi, ovvero esistono delle sfumature tra le due.

Diamo ora una definizione di **classificazione**.

Definizione 2 — Classificazione

Sia data una collezione di dati, possiamo definirma come l'insieme P di M-uple del tipo:

$m_i = (x_{1i}, ..., x_{\text{Mi}}) \in D_1 \times ... \times D_M$

- x_{ji}, indichiamo la feature.
- D_j, indichiamo il dominio di valori per la feature.

L'insieme P può essere aprtizionato in k-classi le cui etichette compongono l'insieme $L = (A_1, ..., A_k)$ Ciascun dato appartiene ad una ed una sola classe, quindi per ogni dato abbiamo una sola etichetta.

Definizione 3 — Algoritmo di classificazione

Un **algoritmo di classificazione** è una funzione computabile $f : P \to L$:

$f(m \in P) = f(x_1, ..., x_m) \in L$

Il nostro obiettivo è capire la forma di questa funzione e di conseguenza quali sono i parametri che la definiscono.

Un classificatore ha successo (in inglese -> hit) su un dato se $f(m)$ coincide con l'etichetta della classe di cui m appartiene realmente. In caso contrario abbiamo un fallimento, o miss.

Una domanda importante che dobbiamo farci è: *Che tasso di errore è considerado accettabile per noi?*

- 50%? Il modello ha imparato a predirre a *caso* i dati che ha imparato.
- 100%? Rientriamo nel caso dell'overfitting che vogliamo sempre evitare.

In linea generale un tasso di errore tra l'1% e il 20% è ritenuto accettabile, tenendo in considerazione anche il contesto in cui stiamo lavorando. Per applicazioni industriali un tasso di errore del 5% fino al 10% è ritenuto accettabile. Per applicazioni mediche invece un tasso di errore superiore allo 0.5% è ritenuto **NON** accettabile.

Quindi il tasso di errore è fortemente influenzato dal contesto su cui vogliamo lavorare, e non è una semplice scelta *netta*.

Se dovessimo avere un modello per una classificazione binaria, con un tasso di errore del 100%, possiamo ritenerlo un buon modello per i nostri scopi? Teoricamente è un modello con un tasso di accuratezza del 100%, ci basta «invertire» il risultato del classificatore.

2.2 Classificazione Iris di Fisher

È un esempio di classificazione fatto dal matematico Fisher nel 1916, raccolse 150 iris, 50 erano di specie setosa, 50 di specie virginica e 50 di specie speciosa.

- 150 fiori iris -> è quello che oggi consideriamo il dataset
- Il dataset è bilanciato -> lo vedremo più avanti nel corso.

Fisher scelse 4 misure (feature) quantitative:
- Lunghezza del petalo
- Larghezza del petalo
- Lunghezza del sepalo
- Larghezza sepalo

Il problema che Fisher voleva studiare e verificare era:
- Son necessarie le 4 misure scelte?
- Le 4 misure scelte, erano **indipendenti** tra loro? Una determinata misura di una implica una determianta dimensione per un'altra?

> 📝 **Possibile domanda d'esame**
>
> Una possibile domanda aperta all'esame può essere:
>
> Perché non possiamo considerare più attuale il dataset utilizzato da Fisher per la sua classificazione?

Esistono modelli descrittivi e predittivi, per esempio quelli predittivi sono usati per prevedere le orbite dei pianeti nello spazio. Gli alberi di decisione sono un tipo di modello descrittivo.

Possiamo avere anche modelli probabilistici che ci restituiscono una percentuale di probabilità sull'appartenenza dell'input a una determinata classe.

> **Attenzione**
>
> Teniamo a mente che la correlazione tra due eventi, non implica che uno sia la causa dell'altro. Questo è importante quando realizziamo dei classificatori, occorre studiare il problema che vogliamo affrontare in modo da evitare queste imprecisazioni.

2.3 Algoritmi di apprendimento

Se abbiamo un classificatore, vogliamo che *impari* a riconoscere le feature che abbiamo scelto per determinati oggetti, quindi come fa il classificatore ad imparare cosa deve fare?

Per esempio noi umani impariamo con gli errori, se commettiamo un errore riusciamo a capire che quello che abbiamo fatto è sbagliato e cercheremo di fare qualcosa di diverso la prossima volta. Per esempio un neonato che impara a camminare commeterà molti errori all'inizio finché non riuscirà a stare all'in piedi e a camminare in autonomia. Stessa cosa i nostri modelli e classificatori, imparano attraverso gli errori.

Commettiamo errori finché non riusciamo a trovare la soluzione corretta al nostro problema.

Non tutti gli algoritmi hanno questa caratteristica, questi che la sfruttano si chiamano algoritmi di addestramento per rinforzo. Per esempio gli alberi di decisione non hanno questa caratteristica.

2.3.1 Contare gli errori

Un problema da risolvere è capire come contare gli errori del modello. Possiamo usare una **matrice di confusione** che riporta in percentuale sulle righeq uanti elementi della classe corrispondente a quella riga sono stati assegnati alle varie classi presenti nella popolazione.

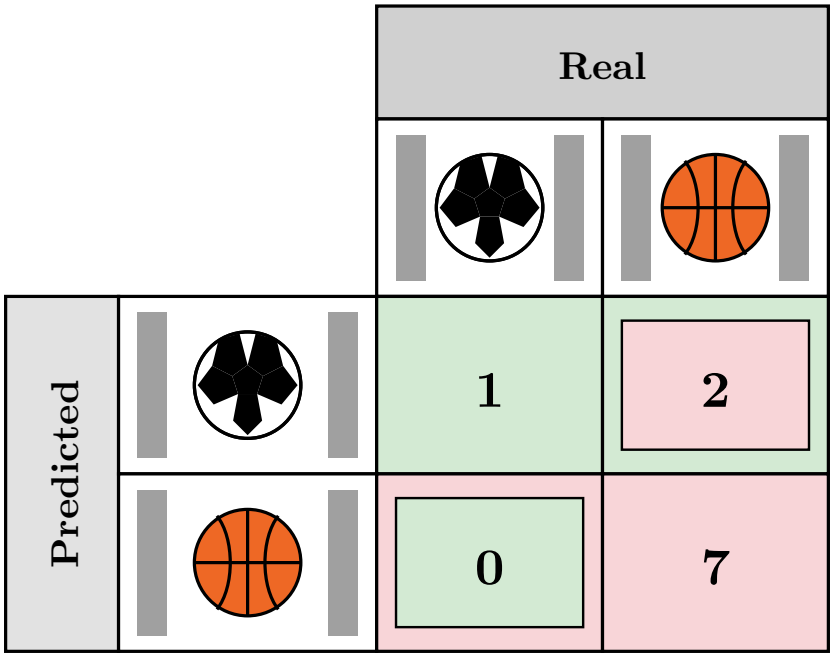


Figura 4. Matrice di confusione

Nell'immagine qui sopra possiamo vedere un esempio di matrice di confusione. Abbiamo un modello addestramento a riconoscere tra palle da calcio e palle da basket. Sulle righe abbiamo la classe predetta dal classificatore, mentre sulle colonne la classificazione reale.

Per i palloni da calcio possiamo vedere che il modello ha predetto correttamente 1 volta il pallone da calcio (vero positivo); mentre per i palloni da basket abbiamo 2 falsi positivi: vuol dire che il classificatore ha classificato il pallone da basket come un pallone da calcio per 2 volte.

Per la predizione sui palloni da basket abbiamo 0 falsi positivi per i palloni da calcio, e 7 veri positivi per i palloni da basket; quindi il modello non ha commesso errori per quanto riguarda la predizione sui palloni da basket.

> **Attenzione**
>
> La matrice di confusione è un concetto fondamentale da apprendere poiché ci indica le prestazioni del modello ed è facile fare confusione quando la si legge per la prima volta.
>
> *Il gioco di parole non è voluto.*

2.3.2 Costruzione di un classificatore

La classificazione è un operazione perfetta per un computer, possiamo elaborare enormi quantità di dati per estrarne informazioni, correlazioni e regolarità.

Vediamo uno schema generale per la costruzione di un modello di classificazione:

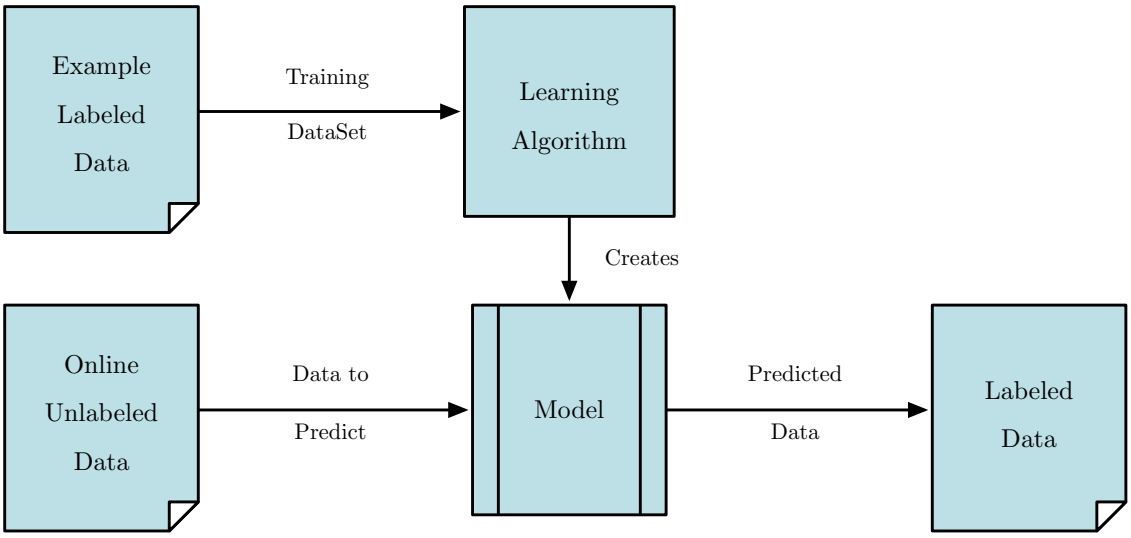


Figura 5. Schema costruzione di un classificatore

Dallo schema possiamo vedere che partendo da un dataset già etichettato, tramite un algoritmo di apprendimento, riusciamo a creare un classificatore. Una volta ottenuto il nostro classificatore per verificare le sue prestazioni dobbiamo dargli in input dati non classificati che il classificatore non ha mai visto durante la fase di addestramento e verificare che dia la risposta corretta.

2.3.3 Fasi di un sistema di classificazione automatica

Le fasi per un sistema di classificazione automatica sono:

1. **Sensing** o **sampling**: Raccolta dei dati dal mondo fisico e traduzione in informazioni digitali; (fondamentale capire quali misure prendere in considreazione)
2. **Segmentazione**: Partizionamento dei dati in unità significative, miglioramento della qualità dei dati, eliminazione di rumore.
3. **Estrazione delle feature**: partendo dai dati segmentati. È un'ottima idea scegliere feature che siano **INVARIANTI alle trasformazioni**; per esempio cogliere informazioni da un ambiente controllato da cui le rileviamo (temperatura, luce).
4. **Classificazione**: Esecuzione dell'algoritmo di assegnazione delle label o classi.
5. **Post-processing**: Valutazione della qualità della classificazione, costi ed errore di classificazione.
6. Decidere se utilizzare il classificatore in un problema reale.

Durante la progettazione di questi classificatori, tipicamente sono più iterazioni di design, ci sono degli aspetti fondamentali da tenere in ordine:

1. **Raccolta dati**: Scegliere i dati su cui lavorare per addestrare il classificatore e conoscenza dei dati su cui dovrà generalizzare.
2. **Selezione delle feature**: Che devono essere prese in considerazione dal nostro classificatore. Abbiamo scelto troppe feature? Troppe poche? Sono rilevanti?
3. **Scelta del modello matematico**: Riuscire a modellare il problema con il linguaggio matematico ci aiuta nella realizzazione del classificatore. Ci permette di formulare ipotesi su come interagiscono le feature tra loro e come si distribuiscono statisticamente.
4. **Training del classificatore**: Addestramento del classificatore sui dati che abbiamo raccolto.
5. **Valutazione**: Possiamo migliorare ancora il modello o abbiamo raggiunto un risultato soddisfacente?

Nota

Piccola digressione fatta dal prof a lezione.

2.4 Perché l'informatica è una scienza?

La domanda del prof a lezione era: *Perché l'informatica è una scienza a differenza della matematica? Che cosa studia l'informatica?*

L'informatica studia si occupa di studiare l'elaborazione, o il calcolo, delle informazioni e delle sue applicaizoni pratiche.

Perché Alan Turing crea la Macchina di Turing?

Lo fa per cercare di replicare il ragionamento del cervello umano, un modello molto semplice rispetto alla complessità del nostro cervello, ma l'intento rimane quello.

Ma l'essere umano è l'unico essere vivente dotato di intelligenza?

Alcuni batteri sono in grado di calcolare; esistono anche altri tipi di intelligenza, come l'intelligenza a sciame

3 Lezione 3 del 28 settembre 2026

Concetti principali: Teoria dell'informazione, distribuzioni, probabilità.

Quanti alberi di decisione possiamo avere per un determinato problema? potenzialmente infiniti.

L'algoritmo per trovare l'albero migliore è NP-completo. è un problema combinatorio. Usiamo delle euristiche per aiutarci nel trovare la soluzione ottimale.

Un evento che è raramente probabile, ci fornisce un'elevata quantità di informazioni.

varianza: quanto mi discosto dalla media, in media. attenzione che la varianza è un termine assoluto, evitiamo indici che dipendono dalle grandezze che misuriamo. Vogliamo indici statistici che ci diano risultati tra $[0; 1]$, perché risultano facilmente interpretabili per noi esseri umani.

Iniziamo con qualche definizione.

Definizione 4 — Variabili e distribuzioni

Data una **variabile** X, indichiamo con $\hat{X}$ il suo intervallo di variabilità, ovvero l'insieme dei vlaori che X può assumere.

Una distribuzione discreta di n occorrenze e k oggetti ci informa su come le n occorrenze si distribuiscano sui k oggetti designati.

Una distribuzione è detta di probabilità se la somma dei valori ad essa assegnata è uguale ad 1.

Ad un possibile evento possiamo associare anche una quantità di informazione:

Definizione 5 — Informazione

L'informazione associata ad un possibile elemento $x \in \hat{X}$ che avviene con probabilità $p(x)$ è data da:

$$I\left(x \in \hat{X}\right) = \log_2 \frac{1}{p(x)} = -\log_2(p(x))$$

Definizione 6 — Sorgente informativa

Una sorgente informativa è la coppia (X, p), dove X è una variabile che assume valori in un determinato insieme e p è una funzione di probabilità che assegna a tutti i suoi possibili valori una determinata probabilità.

Ricordiamo che la probabilità esprime un concetto **a priori**, per esempio ogni faccia di un dado a 6 facce ha 1/6 di probabilità di uscire in un lancio, ammesso che il dado non sia truccato. Mentre la frequenza esprime concetti **a posteriori**, ovvero osserviamo i lanci del dado e vediamo se i risultati rispettano la probabilità teorica.

Un indice statistico che può tornarci utile è l'entropia, definita nel modo seguente:

Definizione 7 — Entropia

L'entropia di una sorgente informativa, ovvero (X, p) si definisce come:

$$H(X,p) = -\sum_{x \in \hat{X}} p(x) \log_2(p(x))$$

- L'entropia raggiunge il valore massimo quando tutti gli elementi sono equiprobabili.
- Se la base del logaritmo è n, allora l'entropia massima è 1.

Iniziamo a parlare di stringhe adesso, seguiranno anche esempi di bioinformatica.

Definizione 8 — Stringa

Dato un alfabeto Γ, una stringa α è una sequenza contigua di caratteri $a_1 a_2 ... a_n$ tale che $a_i \in \Gamma$.

La lunghezza della stringa viene indicata con $|\alpha| = n$.

Con λ, indichiamo la stringa vuota.

Definizione 9 — Sottostringhe

Indichiamo con $\alpha[i]$, con $1 \leq i \leq |\alpha|$, il carattere in posizione i-esima della stringa α.

Indichiamo con $\alpha[i, j]$, con $1 \leq i \leq j \leq |\alpha|$, la sottostringa di lunghezza $j - i + 1$ di α dalla posizione i alla posizione j, entrambe incluse.

- $\alpha[1, i]$ con $1 \leq i \leq |\alpha|$ è un prefisso di α.
- $\alpha[i, |\alpha]\!]$ con $1 \leq i \leq |\alpha|$, è un suffisso di α

Se dovessimo prendere un considerazione un genoma $G \in \Gamma^*$, che possiamo vedere come una stringa, possiamo definire l'insieme dei fattori, ovvero tutte le possibili sottostringhe di G, come:

Definizione 10 — Fattori/Dizionario

Prende il nome di dizionario e viene definito come:

$D(G) = \{G[i, j] : 1 \leq i \leq j \leq |G|\}$

- $D(G)$ si dice dizionario di G.

Possiamo definire anche l'insieme delle posizioni.

Definizione 11 — Occorrenze

Data una parola $\alpha \in D(G)$, l'insieme delle posizioni in cui α occorre in G è dato da:

$$\text{pos}_G(\alpha) = \{i : G[i, j] = \alpha\}$$

Invece per il numero di occorrenze ci serve la molteplicità di α in G.

$$\text{mult}_G(\alpha) = |\text{pos}_G(\alpha)|$$

Possiamo anche definire:

- **hapax**, una parola che occorre una sola volta, $\alpha : \text{mult}_{G(\alpha)} = 1$
- **repeat**, una parola che occorre almeno due volte, $\alpha : \text{mult}_{G(\alpha)} > 1$
- Un repeat è massimale se $\forall x \in \Gamma \Rightarrow \text{mult}_G(\alpha x) = 1$, diventa un hapax.
- Un hapax $\alpha = (a_1, ..., a_{n-1}, a_n)$ è **minimale** se $\text{mult}_G((a_1, ..., a_{n-1})) > 1$
- Un **nullomero** è una parola che non occorre in G, $\alpha : \text{mult}_G(\alpha) = 0 \Rightarrow \alpha \notin D(G)$

Definizione 12 — K-meri

Data una lunghezza di una parola k, i fattori di G lunghi k sono anche detti k-mer.

Possiamo definirlo come:

$D_k(G) = \{\alpha \in D(G) : |\alpha| = k\}$

Passiamo ora a formalizzare le distrubuzioni.

Definizione 13 — Distribuzione

Una distribuzione $\Phi : A \to B$ è una funzione che mappa elementi di A sugli elementi dell'insieme B.

Una distribuzione è discreta se il suo dominio è un insieme discreto.

Definizione 14 — Distribuzione di molteplicità

Una distrubuzione è detta di molteplicità se il suo codominio è l'insieme dei numeri naturali e se essa rappresenta una misura di molteplicità, ovvero una quantità.

Definizione 15 — Distribuzione di frequenza

Una distribuzione si dice di frequenza se il suo codominio è l'insieme dei numeri reali $\mathbb{R}$ e se $\sum_{x \in A} \Phi(A) = 1$

Da una distribuzione di molteplicità, possiamo otterne una di frequenza, trasformiamo le molteplicità in frequenze (gli elemnti di B).

Definizione 16 — Distribuzione di prboabilità

Uan distribuzione di frequenza è detta di probabilità se gli elementi del suo codominio rappresentano delle probabilità.

4 Lezione 4 del 2 ottobre 2026

In questa lezioen si prosegue con richiami di probabilità e statistica.

Abbiamo sempre parlato di sample e di popolazioni, i sample possiamo vederli come dei vettori definiti all'interno di uno spazio vettoriale su cui calcoliamo la distanza tra sample. Altre reti neurali, per esempio le CNN o gli LLM, lavorano su tensori, cioè array multidimensionali che generalizzano il vettore a più indici (ad esempio un'immagine ha rango 3: altezza, larghezza, canali).

Questa definizione vale per i nostri scopi, se fossimo in contesto fisico o matematico puro avrebbe una definizione completamente diversa.

4.1 Indici statistici delle distribuzioni

Vediamo ora quali indici statistici possiamo usare per studiare le distribuzioni. Parleremo di **momenti** di origine m e ordine k di una variabile casuale X è definito come il valore atteso della k-esima potenza dei valori di X.

Definizione 17 — Primo momento, media o valore atteso

La media, o valore atteso è così definita:

$$E[X] = \sum_i p_i x_i$$

Definizione 18 — Secondo momento, varianza

Il secondo momento centrale, chiamato anche varianza, misura la dispersione dei valori della variabile attorno alla media: è il valore atteso del quadrato degli scarti da μ.

$$\text{var}(X) = E[(x-\mu)^2] = \sum_i p^2_{i(x_i-\mu)}$$

- μ è la media.

La varianza possiamo riscriverla anche come:

$$\text{var}(X) = E[X^2] - (E[X])^2$$

Questo può tornarci molto utile se dovessi lavorare su dataset di dimensioni piuttosto elevate permettendoci in una sola passata di calcolare media e varianza.

Definizione 19 — Deviazione standard

La deviazione standard misura la dispersione tipica dei valori attorno alla media, espressa nella stessa unità di misura di X. È definita come la radice quadrata della varianza:

$$\sigma = \text{sd}(X) = \sqrt{\text{var}(X)}$$

Nota

Qual è la differenza tra varianza e deviazione standard?

Guardando le definzioni più o meno raccontano la stessa cosa, quanto ci si discosta dalla media, ma esiste una differenza tra le due. Rappresentano la stessa informazione, ma in modo diverso.

Andando più nel dettaglio:

Varianza:
- L'unità di misura è il quadrato di quella di X.
- Non è direttamente confrontabile con i valori o la media.
- È comoda matematicamente: additiva per variabili indipendenti, $\text{var}(X+Y) = \text{var}(X) + \text{var}(Y)$

Deviazione standard:
- Stessa unità di misura di X.
- Si legge direttamente come: «I valori si disperdono tipicamente di σ attorno alla media.»
- Non è additiva per variabili indipendenti $\text{sd}(X+Y) = \sqrt{\text{sd}(X) + \text{sd}(Y)}$

Esempio: Altezze in cm con $\sigma = 10$ cm: la varianza vale 100cm^2. Dire 100 cm^2 non ti dice subito quanto sono sparse le altezze; dire 10 cm sì, e puoi dire che circa il 68% dei valori sta entro $\mu \pm 10$ (se la distribuzione è gaussiana).

Riassumendo, si usa la varianza per fare i calcoli e la deviazione standard per comunicare e interpretare il risultato. Se σ è grande o piccola dipende solo dal confronto con la scala dei dati (per questo a volte si usa il coefficiente di variazione $\frac{\sigma}{\mu}$).

Definizione 20 — Coefficiente di variazione

È definito come il rapporto tra la deviazione standard e il valore assoluto della media.

$$\frac{\text{sd}(X)}{|\mu|}$$

Ci permette di misurare la dispersione relativa alla scala dei dati. Puo tornarci utile per confrontare la variabilità di feature con scale diverse, per esempio per trovare feature costanti o rumorose.

Definizione 21 — Terzo momento, skewness

La skewness è definita come:

$$\gamma = E[\left(\frac{X-\mu}{\sigma}\right)^3$$

Ci permette di dire dove troviamo la coda più lunga:
- Con skewness > 0 la coda lunga è verso destra, moda $<$ mediana $<$ media
- Con skewness < 0 la coda lunga è a sinistra, media $<$ mediana $<$ moda
- Nel caso in cui la skewness sia 0, moda, media e mediana coincidono.

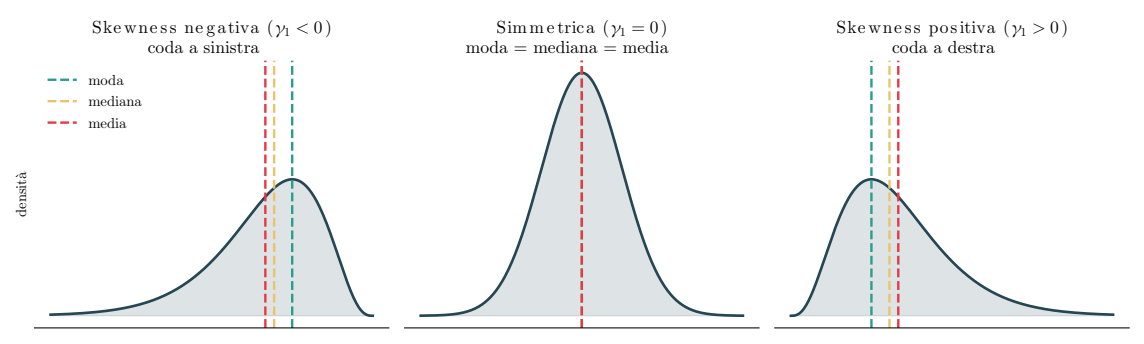


Figura 6. Grafico skewness di una distribuzione

Definizione 22 — Quarto momento, indice di Kurtosis

È un indice di allungamento, o appiattimento della coda della distribuzione.

$$\text{Kurt}(X) = E\left[\left(\frac{X-\mu}{\sigma}\right)^4\right]$$

Ora diamo un'occhiati agli indici di dispersione e di diversità.

Un **indice di dispersione** riassume la misura con la quale i valori di una distrubizione statistica quantitativa sono distanti da un valore centrale (media o mediana).

Se la variabile statistica è di tipo nominale, allora prende il nome di indice di diversità. Una variabile si dice nominale quando il carattere statistico che stiamo studiando assueme stati discreti non ordinabili.

Definizione 23 — Indice di Simpson

L'indice di Simpson è una misura quantitativa che riflette quanti tipi di (specie) diverse ci sono in un dataset (comunità).

La diversità ha tre aspetti principali, che l'indice sintetizza in un unico valore:

- Ricchezza (richness): Il numero di specie diverse.
- Equitabilità: grado di omogeinità con il quale gli individui sono distribuiti nelle varie specie che compongono una comunità.
- Dominanza: Il grado con cui uan specie è dominante verso le altre specie facenti parte della comunità.

È definito come:

$$\lambda = \sum_{i=1}^{R} p_i^2$$

- R è la ricchezza, ovvero il numero di specie distinte nella comunitàe p_i è l'abbondanza della i-esima specie.

Definizione 24 — Indice di Shannon (Entropia di Shannon)

L'indice di Shannon è un indice di diversità basato sulla media pesata dei logaritmi dell'abbondanza proporzionale della specie in una comunità. È definito come:

$$H = -\sum_{i=1}^{R} p_i \ln(p_i)$$

Definizione 25 — Numeri di Hill

I numeri di Hill sono una generalizzazione delle misure di diversità.

Dato un ordine q, la diversità è misurata prendendo in considerazione il reciproco della media generalizzata M_{q-1} dell'abbondanza proporzionale dei tipi nel dataset.

È definita come:

$${}^{q}D = \frac{1}{M_{q-1}} = \frac{1}{\sqrt[q-1]{\sum_{i=1}^{R} p_i \, p_i^{q-1}}} = \left(\sum_{i=1}^{R} p_i^q\right)^{\frac{1}{1-q}}$$

- Per $q = 0$ è la media delle abbondanze.
- Per $q = 1$ non è definità, tuttavia si può calcolare il limite per q che tende a 1 e si ottiene:

$${}^{1}D = \frac{1}{\prod_{i=1}^{R} p_i^{p_i}} = e^{-\sum_{i=1}^{R} p_i \ln(p_i)}$$

- Il logaritmo di ${}^{q}D$ è l'**entropia di Rényi** di ordine q, che generalizza l'entropia di Shannon (ottenuta per $q \neq 1$):

$$\ln({}^{q}D)$$

- Per $q = 2$ equivale all'inverso dell'indice di Simpson:

$${}^{2}D = \frac{1}{\lambda}$$

4.2 Confrontare distribuzioni

Riassumere una distribuzione in un solo indice statistico è svantaggioso se vogliamo confrontare le proprietà di due distribuzioni diverse fra loro. Potremmo utilizzare misure come:

- Similarità tra gli insiemi.
- Vicinanza/prossimità per spazi vettoriali.
- Correlazione tra variabili.
- Divergenza tra distribuzioni di probabilità.

Un modo per calcolare la similiratà tra due insiemi, che abbiamo visto in laboratorio, è la **similarità di Jaccard**.

Definizione 26 — Similarità di Jaccard

Ci dice quanto due insiemi siano simili, ovvero quanti elementi hanno in comune. Se prendiamo l'esempio di due dizionari dei k-meri di due genomi G_1 e G_2 abbiamo:

$$J_k(G_1, G_2) = \frac{|D_k(G_1) \cap D_k(G_2)|}{|D_k(G_1) \cup D_k(G_2)|}$$

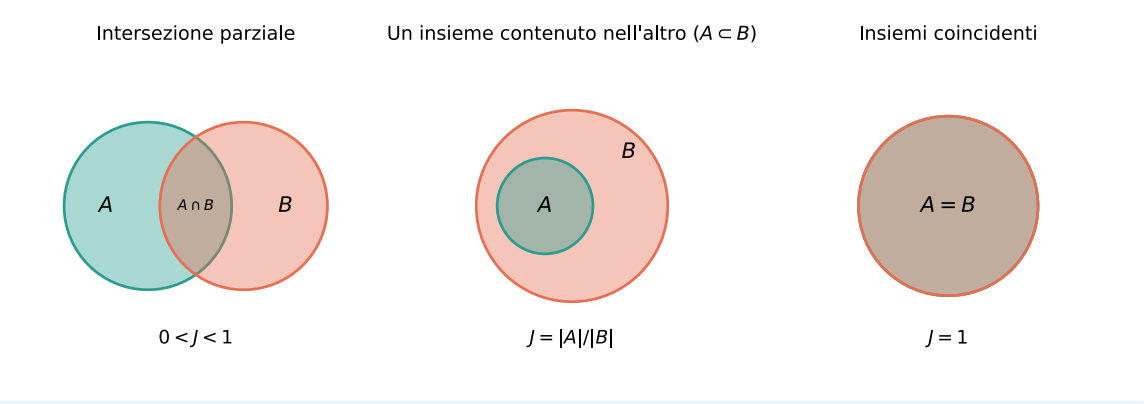


Figura 7. Valori possibili per la similarità di Jaccard

Definizione 27 — Similarità di Jaccard generalizzata, per multi insiemi

Questo ci permette di dire quanto due multi insiemi siano simili, ovvero ci permette di tenere in considerazione anche le molteplicità degli elementi. Sia $D_k(G_1, G_2) = D_k(G_1) \cup D_k(G_2)$, allora:

$$J'_k(G_1, G_2) = \frac{\sum_{\alpha \in D_k(G_1, G_2)} \min\left(\text{mult}_{G_1}(\alpha), \text{mult}_{G_2}(\alpha)\right)}{\sum_{\alpha \in D_k(G_1, G_2)} \max\left(\text{mult}_{G_1}(\alpha), \text{mult}_{G_2}(\alpha)\right)}$$

Un altro modo per confrontare distribuzioni è quello di usare le distanze.

Definizione 28 — Distanza del coseno

Misura il coseno dell'angolo formato dai due vettori in uno spazio n dimensionale. Questo ci restituisce valori compresi in un intervallo tra $[-1 : 1]$.

Prendiamo in considerazione due vettori $A, B \in \mathbb{R}^n$:

$$\text{cos_sim(A, B)} = \frac{<A \mid B>}{<A \mid A> <B \mid B>} = \frac{\sum_{i=1}^{n} A[i]B[i]}{\sqrt{\sum_{i=1}^{n} A[i]^2}\sqrt{\sum_{i=1}^{n} B[i]^2}}$$

- $< \mid >$, indica il prodotto scalare tra i due vettori.

Definizione 29 — Distanza di Minkowski

La distanza di Minkowski è la metrica indotta dalla norma ℓ_k sullo spazio vettoriale normato $\mathbb{R}^n$ (per $k \geq 1$), cioè $d_{k(A,B)} = |A - B|_k$. Uno spazio vettoriale normato è uno spazio vettoriale su cui è definita una norma, ovvero una funzione che assegna a ogni vettore una lunghezza.

Dato $k \in \mathbb{N}^+$ e due vettori $A, B \in \mathbb{R}^n$ è definita come:

$$d_k(A,B) = \left(\sum_{i=1}^{n} |A[i] - B[i]|^k \right)^{\frac{1}{k}} = \sqrt[k]{\sum_{i=1}^{n} |A[i] - B[i]|^k}$$

È la generalizzazione della distanza euclidea, per $k = 2$ e della distanza di Manhattan quando $k = 1$.

Definizione 30 — Correlazione di Pearson

La correlazione di Pearson misura il grado di dipendenza **lineare** tra due variabili reali, cioè quanto i loro scostamenti dalla media tendono a concordare. Essa è definita come:

$$\rho(X,Y) = \frac{\text{cov}(X,Y)}{\sigma_X \, \sigma_Y}$$

- ρ assume valori compresi nell'intervallo $[-1 : 1]$.
- Se $\rho = 1$, le due variabili sono correlate in modo crescente.
- Se $\rho = -1$, le due variabili sono correlate in modo decrescente.
- Se $\rho = 0$, le due variabili non sono in correlazione tra loro.

dove $\text{cov}(X,Y)$ è la covarianza tra le due variabili X e Y, definita come il valore atteso del prodotto delle deviazioni dalle rispettive medie. Su n osservazioni si stima con la media dei prodotti degli scarti:

$$\text{cov}(X,Y) = E[(X - E[X])(Y - E[Y])] = \tfrac{1}{n} \textstyle\sum_{i=1}^{n} (x_i - \overline{x})(y_i - \overline{y})$$

Definizione 31 — Correlazione di Spearman

È la correlazione di Pearson calcolata sui ranghi anziché sui valori: al posto di x_i si usa $R(x_i)$, la posizione di x_i nell'ordinamento dei valori di X. Misura quindi la dipendenza monotòna tra le variabili.

Definizione 32 — Correlazione di Kendall

Considera tutte le coppie di osservazioni (i, j) e conta quante sono concordanti (le due variabili ordinano i punti nello stesso modo) e quante discordanti (inversioni di rango):

$$r = \frac{C - D}{\binom{n}{2}} = \frac{C - D}{\frac{n(n-1)}{2}}$$

dove C è il numero di coppie concordanti, D quello delle discordanti e $\binom{n}{2} = \frac{n(n-1)}{2}$ il numero totale di coppie.

5 Lezione 5 del 5 ottobre 2026

Riprendiamo il discorso su come confrontare due distribuzioni di probabilità. Abbiamo visto che abbiamo diversi modi per misuarne la differenza come:

- Similarità tra gli insiemi
- Vicinanza o prossimità per spazi vettoriali
- Correlazione tra variabili
- Divirgenza tra distribuzioni di probabilità

Qui andiamo avanti con le divergenze tra distribuzioni di probabilità.

Partiamo definendo la **divergenza di Kullback-Leibler** che ci permette di misurare più informazioni per due distribuzioni.

Definizione 33 — Divergenza Kullback-Leibler

Prese due distribuzioni di probabilità, P e Q, **definite sullo stesso dominio D**, la divergenza di Kullback-Leibler di P da Q ci informa sul guadagno di informazione nell'utilizzare P piuttosto che Q.

$$\mathrm{KL}(P\|Q) = \sum_{x\in D} P(x)\log\left(\frac{P(X)}{Q(X)}\right)$$

- $(P\|Q)$, si legge da P a Q.
- Non è simmetrica: $\mathrm{KL}(P\|Q) \neq \mathrm{KL}(Q\|P)$.
- $\mathrm{KL} \geq 0$, questo vale sempre.

Possiamo avere una nozione di divergenza simmetrica con **Jensen-Shannon**.

Definizione 34 — Divergenza di Jensen-Shannon

È una nozione di divergenza **simmetrica** basata sulla distribuzione media $A = \frac{P+Q}{2}$ e non sulla media delle divergenze.

$$\mathrm{JSD}(P,Q) = \frac{\mathrm{KL}(P\|A) + \mathrm{KL}(Q\|A)}{2}$$

Ha un valore massimo di 1 se la base del logaritmo è 2.

Queste divergenze che abbiamo appena visto fanno parte di una famiglia di misure chiamate **f-divergenze**, il cuo scopo è quello di rappresentare la divergenza come media dell'odds ratio (rapporti di probabilità) ponderata da una funzione f.

Definizione 35 — Divergenza di Hellinger

È una importante f-divergenza definita come:

$$\mathrm{HE}(P,Q) = \frac{1}{\sqrt{2}}\sqrt{\sum_{x\in D}\left(\sqrt{P(x)} - \sqrt{Q(x)}\right)^2}$$

Può essere riscritta anche come:

$$\mathrm{HE}^2(P,Q) = 1 - \sum_{x\in D}\sqrt{P(x)Q(x)}$$

I valori di essa sono compresi tra $[0...1]$

Nota

A questo punto il prof si è lasciato andare su una digressione filosofica su cosa sia l'entropia. Non so se metterla in questa appunti oppure lasciare solo questa nota che rompe la quarta parete.

Una cosa fondamentale della digressione del prof, è che se misuro l'entropia su 3 sistemi **diversi** non posso confrontare le entropie tra di loro.

Alla fine, da come si può vedere, ho messo un sunto del discorso. Torniamo ai nostri appunti...

Dopo la digressione sull'entropia, servita per introdurre la cross-entropy che vedremo definita tra poco. Abbiamo iniziato a parlare di distribuzioni di probabilità congiunte, entropia congiunta e cross-entropy.

Iniziamo definendo cos'è una distribuzione di probabilità congiunta.

Definizione 36 — Distribuzione di probabilità congiunta

Siano X e Y, due variabili aleatorie, e sia $p(x,y)$ la loro **probabilità congiunta** dei due eventi $x \in \hat{X}$ e $y \in \hat{Y}$.

La distribuzione di probabilità congiuntia di X e Y è data da:

$$p_{X,Y} = \left(p(x,y) \mid x \in \hat{X}, y \in \hat{Y}\right)$$

è definita sul prodotto cartesiano $\hat{X} \times \hat{Y}$.

Possiamo definire l'entropia congiunta per una sorgente informativa.

Definizione 37 — Entropia congiunta

Data la sorgente informativa $\left(X \times Y, p_{X,Y}\right)$ definiamo **entropia congiunta** come l'entropia di tale sorgente informativa:

$$H\left(\left(X \times Y, p_{X,Y}\right)\right) = -\sum -\left(x \in \hat{X}, y \in \hat{Y}\right) p(x,y)\log_2(p(x,y))$$

Definizione 38 — Informazione mutua

L'informazione mutua $I(X,Y)$ è data dalla divergenza entropica delle distribuzioni p_x e p_y.

Arriviamo in fine alla definizione di cross-entropy.

Definizione 39 — Cross-entropia

Data una variabile aleatoria X e due distribuzioni di probabilità discrete, P e Q, definite su di essa, la cross-entropy è definita come:

$$H(P\|Q) = -E_{p[\log_2(Q)]} = -\sum_{x\in\hat{X}} P(x)\log_2(Q(x)) = H(P) + \mathrm{KL}(P\|Q)$$

misura il numero medio di bit necessari per identificare un evento estratto da $\hat{X}$ nel caso sia utilizzato uno schema ottimizzato per uan distribuzione di probabilità Q piuttosto che per la distribuzione P.

5.1 Classificazione: tecniche di validazione

Ora iniziamo a dare un'occhiata alle verie tecniche di validazione per i nostri modelli di intelligenza artificiale.

Noi possiamo costruire due modelli per «classificare» i dati secondo due tecniche principali:

- **Regressione**: si costruisce un modello per predirre un determianto valore in output partendo da input numerici, si cerca di predirre se l'output stia sotto o sopra a una certa curva.
- **Classificazione**: Prendiamo un oggetto in input e cerchiamo di capire a quale classe appartiene tra le classi che ci interessano.

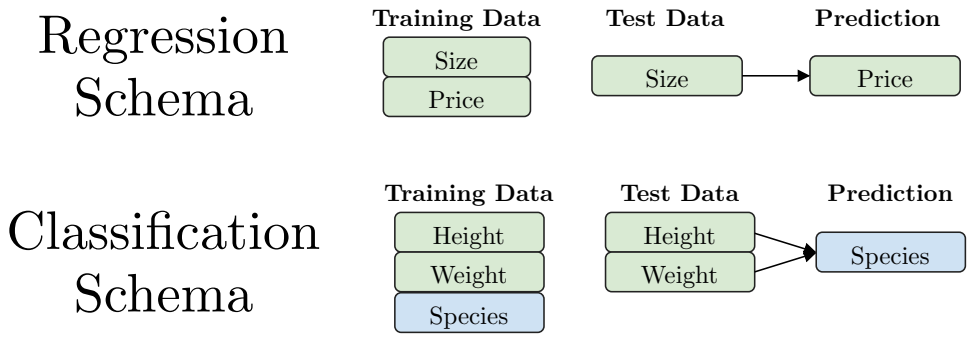


Figura 8. Schema regressione e classificazione.

A seconda del modello che abbiamo realizzato, uno che lavora tramite regressione o uno che lavora tramite classificazione, avremo diversi metodi per valutarne le prestazioni.

La tecnica di base per valutare le prestazioni di un modello di classificazione è quello di prendere il nostro dataset e dividerlo in due dataset diversi, uno di **training**, utilizzato per la fase di addestramento, e uno di **test** per testare le prestazioni del modello una volta terminata la fase di addestramento. La divisione avviene secondo una certa **percentuale**. Di solito si fa un 70% training e 30% test, o anche 80% training e 20% test. Però le domande più importanti da chiedersi è come suddividere questo dataset? Che dati metto nel training set e test set? Ho la stessa distribuzione delle classi da classificare in entrambi i dataset?

Altre domande che dobbiamo porci sono: Il classificatore valuta in maniera bilanciata tutte le label? Ha delle preferenze per una certa classe? Se le ha da cosa dipendono?

Per valutare i nostri modelli utilizzeremo, *ovviamente*, degli indici matematici, per esempio:

- **Accuratezza**: la proporzione di predizioni corrette rispetto al totale delle osservazioni analizzate.
- **Robustezza**: capacità del modello di manipolare dati con rumore o informazioni mancanti.
- **Velocità**:
  - Tempo necessario per costruire il modello, durata del training, è la fase più dispendiosa.
  - Tempo necessario per usare il modello.
- **Scalabilità**: efficienza su dati presenti in memoria secondaria.
- **Interpretabilità** dei risultati.

Prendiamo in esempio questo grafico che ci mostra le prestazioni di un classificatore per due classi A e B.

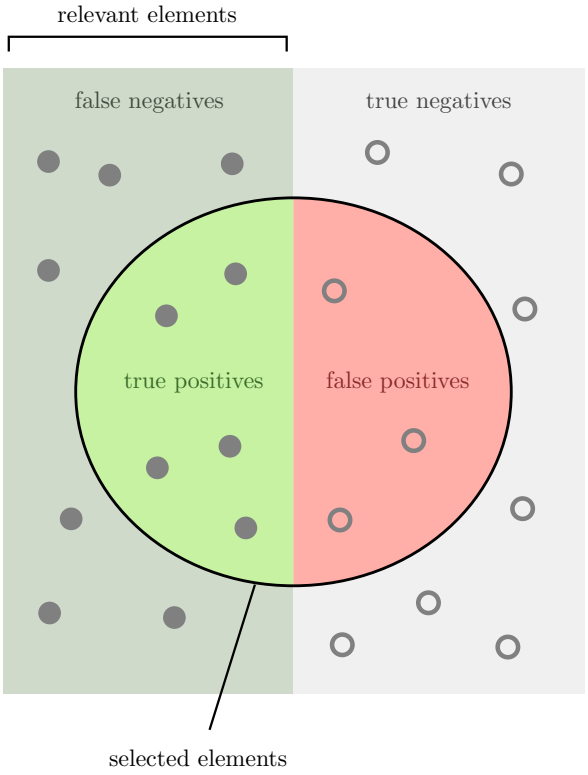


Figura 9. Precision e recall tra due classi.

A sinistra abbiamo gli elementi della classe A e a destra quelli della classe B, ipotizziamo di assegnare alla classe A il valore di verità ***true***. A sinsitra abbiamo gli elementi rilevanti, ovvero gli elementi che abbiamo deciso che saranno i **true positive (TP)** nel caso in cui il classificatore li classifichi correttamente, abbiamo poi i **false negative (FN)**, dove il classificatore ha classificato erroneamente elementi di A come elementi di B.

Nella parte destra del diagramma abbiamo i **false positive (FP)**, elementi di B che il classificatore ha riconosciuto come tali. Abbiamo poi i **true negative (TN)**, elementi di B dove il classificatore li ha erroneamente classificati come elementi di A.

Definiamo formalmente **precision** e **recall**:

Definizione 40 — Precision e recall

Definiamo come **precision** la frazione di elementi rilevanti tra tutti gli elementi classificati come classe A:

$$Precision = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Definiamo, invece, come **recall** la frazione di elementi classificati come classe A tra tutti gli elementi rilevanti.

$$Recall = \frac{\text{TP}}{+ \text{TP} + \text{FN}}$$

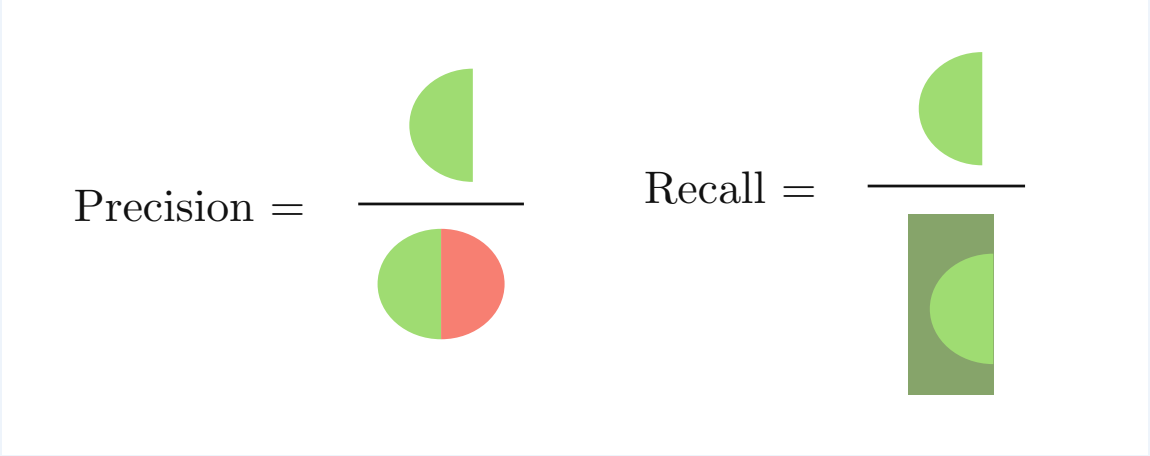


Figura 10. Precision e recall.

Entrambi hanno valori tra 0 e 1.

Più alti saranno i valori di **precision** e **recall** e più sarà accurato il nostro classificatore. In certi contesti ci conviene utilizzare un solo indicatore, evitando di doverne utilizzare due. Per questo si usa l'$\boldsymbol{F_1}$**-score**.

Definizione 41 — F_1-score

L' **F_1**-score è una misura che combina precision e recall facendone una media armonica, è definita come:

$$F_1 = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\ \text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$$

Come **precision** e **recall**, l'F_1-score ha valori tra 0 e 1. Più è alto il valore e più la bontà del modello è alta, occhio però a valori troppo vicini ad 1, potrebbero indicare un potenziale **overfitting**.

Parliamo ora nello specifico dell'**accuracy** del modello. *Accuratezza e accuracy vengono usati in modo intercambiabile.*

Definizione 42 — Accuracy (Accuratezza)

L'**accuratezza**, in inglese **accuracy**, misura la quantità di oggetti correttamente classificati rispetto al totale degli oggetti da classificare, ovvero:

$$Accuracy = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Nota

Occhio a chi decidiamo chi è **true positive** e chi è **true negative**, potremmo ottenere risultati completamente diversi. Dobbiamo sempre tenere sempre in considerazione **cosa** vogliamo misurare come valore **true**. Magari per me il valore **true** è se un paziente è malato, per un'altra implementazione sullo stesso problema di classificazione può rappresentare il paziente sano.

L'accuratezza di per se non cambia, dato che **TP** e **TN** vengono sommate entrambe a numeratore, ma concenttualmente, se prediamo in considerazione l'esempio di paziente malato o sano, sono cose completamete diverse.

Questo vale se abbiamo dataset bilanciati, nel caso in cui dovessimo avere dataset **sbilanciati** dobbiamo usare un altro tipo di accuratezza per valutare il nostro classificatore.

Definizione 43 — Accuracy per dataset sbilanciati

Se abbiamo un dasaet con 95 campioni negativi, e 5 positivi, un classificatore che assegna la classe negativa avrà come accuracy il 95%. Vediamo come calcolare l'accuracy per dataset sbilanciati.

$$Balanced\ accuracy = \frac{\text{TPR} + \text{TNR}}{2}$$

- **TPR (True Positive Rate)**: è dato da $TPR = \frac{\text{TP}}{\text{TP} + \text{FN}}$, è detta anche sensitività, perché cattura quanto il classificatore è sensibile nel riconoscere una classe.
- **TNR (True Negative Rate)**: è dato da $TNR = \frac{\text{TN}}{\text{TN} + \text{FP}}$ detta anche specificità perché cattura quanto il classificatore è specifico nel distinguere la classe dalle altre.

Un altro modo grafico per rappresentare i risultati ottenuti dal classificatore è quello di usare una matrice di confusione. Per esempio:

		Actual class		
		Positive	Negative	
Predicted class	Positive	**TP:** True Positive	**FP:** False Positive (Type I Error)	**Precision:** $\frac{\textbf{TP}}{\textbf{(TP + FP)}}$
	Negative	**FN:** False Negative (Type II Error)	**TN:** True Negative	**Negative Predictive Value:** $\frac{\textbf{TN}}{\textbf{(TN+FN)}}$
		Recall or Sensitivity: $\frac{\textbf{TP}}{\textbf{(TP + FN)}}$	**Specificity:** $\frac{\textbf{TN}}{\textbf{(TN + FP)}}$	**Accuracy:** $\frac{\textbf{TP + TN}}{\textbf{(TP + TN + FP + FN)}}$

Figura 11. Matrice di confusione.

Precisiamo che: sensitività, precisione, specificità e NPR sono specifiche della classe presa in considerazione, mentre l'accuracy è invariate rispetto alla classe come già detto.

Domanda, se avessimo 3 o più classi? In tal caso faremo una matrice di confusione $n \times n$, dove n è il numero di classi. Poi per ogni classe calcoliamo i valori **TP**, **FP** e **FN**.

Un'altra metrica utile è la **AUC** (Area Under the Curve)

Definizione 44 — AUC (Area Under the Curve

Prendiamo in esame il seguente grafico:

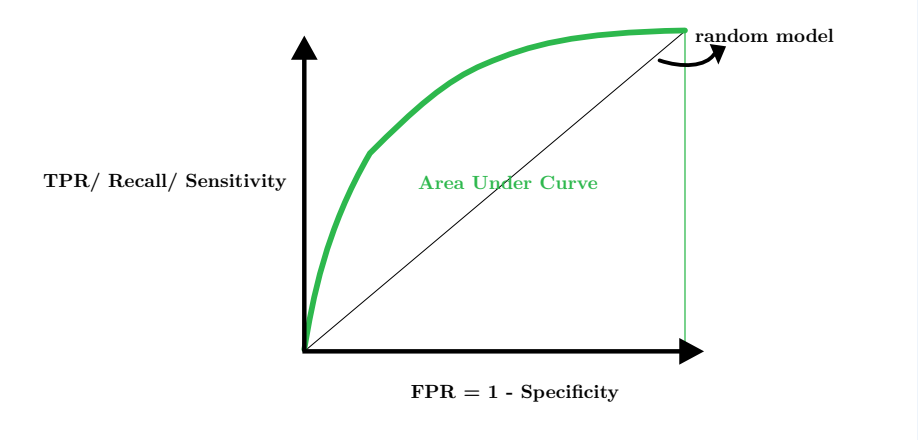


Figura 12. Area Under the Curve.

Prendiamo una curva di riferimento, in questo caso la retta passante per l'origine e indichiamo con la curva verde le prestazioni del nostro classificatore. Più la curva verde è vicina alla curva di riferimento è più indica che il modello classifica casualmente i valori che gli diamo in input.

Vogliamo che la curva verde vada quasi a formare un angolo retto, ovviamente evitado di andare in overfitting.

Nota

Nel caso in cui dovessimo ottenere un modello con un indice delle prestazioni del 0.95 al primo colpo, siamo tutti felici e contenti. Ma nel caso in cui dovessimo ottenere un modello con prestazioni discrete, circa dello 0.70, non vuol dire che sia necessariamente un modello orrendo, *ovviamente il risultato non è bellissimo*, ma adando a studiare i dati di partenza possiamo capire come migliorare il modello per aumentarne le prestazioni.

Molto probabilmente ci sarà qualche errore nostro a monte durante la fase di design, o abbiamo interpretato male i dati, che con un cambio implementativo sul modello non ci permetterà di ottenere risultati migliori.

Altri indicatori per misurare le pretazioni del nostro modello sono:

Definizione 45 — Micro average

La **micro average** prende il nome dal fatto che riassume le prestazioni per **ogni campione** del dataset. Quindi formalmente è definita come:

$$Precision_{\text{micro}} = \frac{\sum_{i=1}^{|G|} \text{TP}_i}{\sum_{i=1}^{|G|} (\text{TP}_i + \text{FP}_i)}$$

$$Recall_{\text{micro}} = \frac{\sum_{i=1}^{|G|} \text{TP}_i}{\sum_{i=1}^{|G|} (\text{TP}_i + \text{FN}_i)}$$

$$F_{1_{\text{micro}}} = 2\frac{\text{P}_{\text{micro}} \cdot \text{R}_{\text{micro}}}{\text{P}_{\text{micro}} + \text{R}_{\text{micro}}}$$

Questi indicatori posso risultare fuorvianti per dataset con classi sbilanciate, quindi tenere sempre in considerazione il dataset su cui stiamo lavorando.

Definizione 46 — Macro average

La **macro average** prende il nome dal fatto che fa la media su gruppi più grandi, ovvero **valuta le prestazioni per le singole classi** invece che sui singoli campioni:

$$Precision_{\text{micro}} = \frac{1}{|G|} \sum_{i=1}^{|G|} \frac{\text{TP}_i}{\sum_{i=1}^{|G|} (\text{TP}_i + \text{FP}_i)} = \frac{\sum_{i=1}^{|G|} \text{P}_i}{|G|}$$

$$Precision_{\text{micro}} = \frac{1}{|G|} \sum_{i=1}^{|G|} \frac{\text{TP}_i}{\sum_{i=1}^{|G|} (\text{TP}_i + \text{FN}_i)} = \frac{\sum_{i=1}^{|G|} \text{R}_i}{|G|}$$

$$F_{1_{\text{micro}}} = 2\frac{\text{P}_{\text{micro}} \cdot \text{R}_{\text{micro}}}{\text{P}_{\text{micro}} + \text{R}_{\text{micro}}}$$

Se otteniamo dei valori alti vuol dire che il nostro classificatore funziona bene, in media, per ogni singola classe. La macro media si addice di più per dati con una distribuzione per classi sbilanciata.

5.2 Cross-validation

Una tecnica importante per validare modelli statistici è la **cross-validation**, ci permette di capire come i risultati di un'analisi statistica si generalizzeranno a un insieme di dati indipendente.

Ha come obiettivo pricipale di testare le capacità del modello di prevedere nuovi dati che non sono stati utilizzati durante la fase di addestramento o durante la sua creazione.

Ci aiuta anche a valutare eventuali problemi di **overfitting** o di **selection bias**.

Il selection bias avviene quando il dataset che prendiamo in considerazione è viziata da fattori non ogettivi, per esempio se chi crea un dataset vuole fare in modo di discriminare una determinata classe da classificare può farlo manipolando i dati.

La **cross-validation** si suddivide in:

- **Esaustiva**: addestra e verifica il modello su tutti i possibili modi di dividere il dataset in training set e test set.

- **Non esaustiva**: solo un sotto insieme dei possibili modi di divisione viene preso in considerazione. Generalmente vengono scelti casualmente dal dataset di partenza.

Definizione 47 — Cross-validation non esaustiva

La tecnica di cross-validation non esaustiva più usata è la **k-fold cross-validation**. In questa tecnica il dataset viene diviso i k parti di dimensioni uguali. A turno ogni parte viene usata come test set metnre il resto come training set.

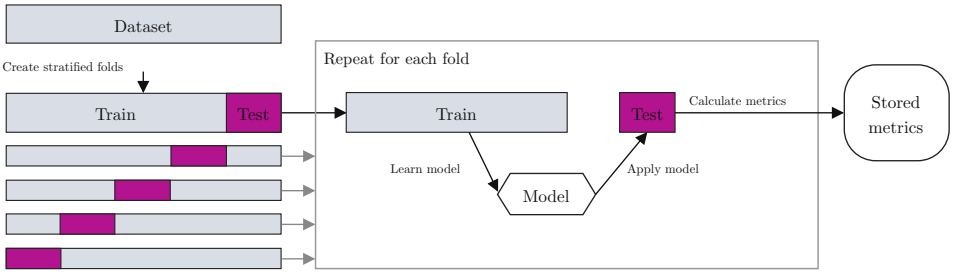


Figura 13. Esempio di cross-validation k-fold.

Definizione 48 — Cross-validation non esaustiva – Stratified k-folding

È una variante del k-fold che restituisce fold stratificati. I fold vengono creati **preservando** la percentuale di campioni per ciascuna classe presente nel dataset, in un contesto di classificazione binaria o multiclasse.

Se il dataset è sbilanciato, è **fondametale rispettare lo sbilanciamento anche nel test e training set** infatti questa tecnica è particolarmente indicata per dataset sbilanciati.

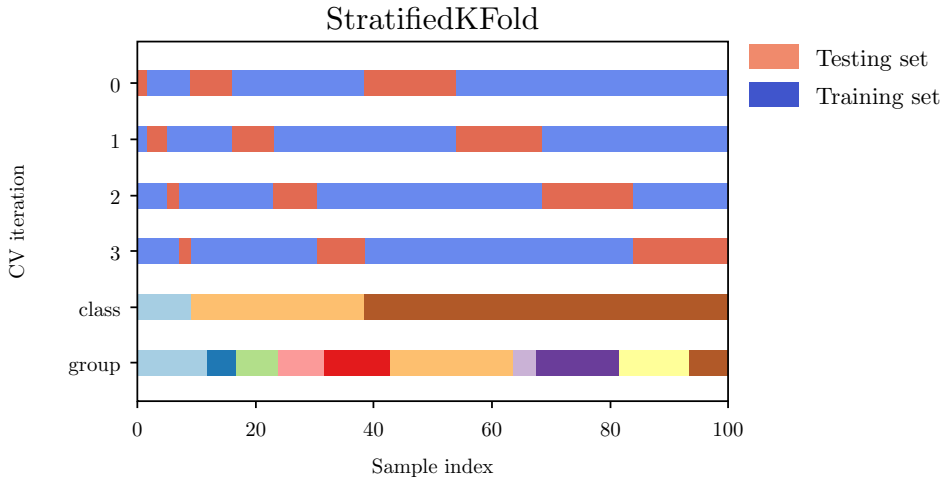


Figura 14. Esempio di stratified k-fold.

6 Lezione 6 del 9 ottobre 2026

La lezione 6 si concentra sugli algoritmi di classificazione.

6.1 Algoritmi di classificazione

I modelli di classificazione possono essere divisi in due tipologie principali:

- ***Eager***: costruiscono un modello generale ed indipendente dall'input partendo dai dati di addestramento.
- ***Lazy***: memorizzano i dati di addetramento in forma grezza, il processo di classificazione è rimandato all'arrivo di una nuova istanza da classificare.

I modelli lazy richiedono meno tempo in fase di addestramento, ma aumenta il tempo durante la fase di classificazione. I modelli lazy però hanno uno spazio delle ipotesi molto più ricco. Possono usare in modo più diretto funzioni locali di classificazione che forniscono delle approssimazioni a livello globale.

I modelli eager invece fanno riferimento a una sola ipotesi per coprire l'universo delle osservazioni.

Vedremo il modello K-nearest neighbor come approccio di classificazione lazy.

6.1.1 K-NN, K-nearest neighbors (Lazy)

È un algoritmo di classificazione lazy.

I data point sono rappresentati in uno spazio n dimensionale. Ogni dimensione può avere valori reali o discreti.

Su questo spazio calcoliamo le distanze tra i vari punti cercando di stabilire a quale classe appartiene il punto che ci interessa calcolando la distanza dai suoi k vicini.

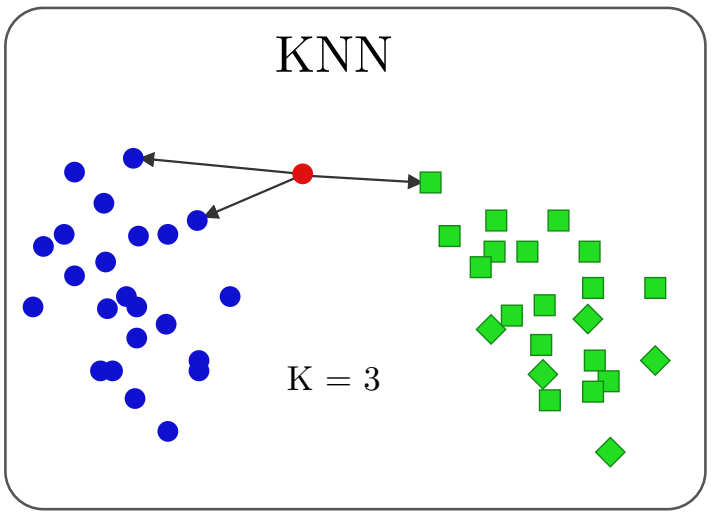


Figura 15. Esempio di K-NN.

Ricapitolando, dato un training set e un'istanza da classificare, l'algoritmo assegna a tale istanza la classe che più è rappresentata tra i suoi k vicini più prossimi. L'algoritmo dipende molto dal parametro k, di solito si usano valori dispari.

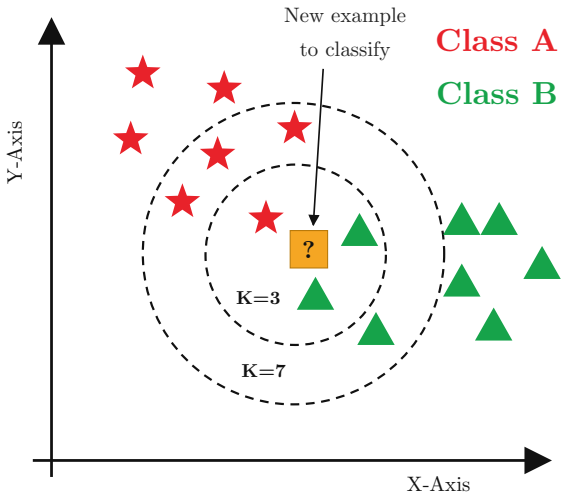


Figura 16. K-NN al variare di k.

K-NN non fa alcuna ipotesi sulla possibile distribuzione dei dati che vengono analizzati, di conseguenza è una delle prime scelte quando c'è poca o nessuna conoscenza preliminare sulla distribuzione dei dati.

Nel caso di dataset sbilanciati K-NN tende a favorire la classe più numerosa, dato che ha più occorrenze nel dataset che stiamo studiando. Se dovessimo avere un dataset con 1000 campioni, 800 di classe A e 200 di classe B, è altamente probabile che un elemento venga classificato come elemento di classe A.

6.1.2 K-NN pesato

Per migliorare le prestazioni di K-NN con dataset sbilanciati si può utilizzare una versione dove le classi dei vicini sono pesate in base alla loro distanza.

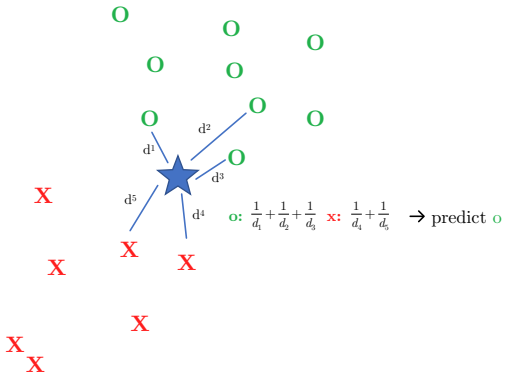


Figura 17. K-NN pesato.

Nei dataset sbilanciati si può pensare di pesare anche l'apporto di ogni vicino con l'inverso del numero totale di punti presenti nel dataset ed aventi al stessa classe del vicino in questione.

6.1.3 Alberi di decisione (Eager)

Un albero di decisione descrive un albero di ricerca n-ario dove i nodi foglia rappresentano le classificazioni e le ramificazioni l'insieme delle proprietà che portano a quelle classificazioni. Ogni nodo interno contiene un test che stabilisce quale sotto-albero esplorare.

Un test valuta una feature o una combinazione di feature.

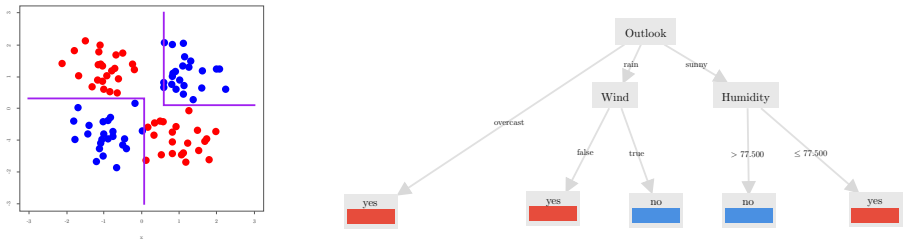


Figura 18. Albero di decisione.

6.1.3.1 Costruzione dell'albero

Vediamo come costruire un albero di decisione:

Partiamo da un nodo radice che racchiude la totalità dei dati di addestramento e si segue l'algoritmo:

1. Se tutti i dati di training che sono all'interno di un nodo t assumono lo stesso valore sull'attributo classificatore, creare un nodo foglia contenente tutti i dati.
2. Altrimenti, mediante una misura di goodness effettuare una partizione S degli elementi rappresentati dal nodo
3. Si sceglie la partizione S^* che ad esempio massimizza la misura di goodness.
4. Si creano tanti figli del nodo t quante sono le classi presenti in S^*.
5. Un nodo figlio è detto puro se tutti i suoi elementi assumono lo stesso valore sull'attributo classificatore.
6. Si applicano ricorsivamente i passi precedenti sui nodi impuri o che non rappresentano una singola classe secondo una soglia di maggioranza.

Il nostro obiettivo è trovare l'albero più semplice a parità di prestazioni, ottenendo un albero più compatto possibile.

Teniamo in considerazione che la ricerca dell'albero migliore è un problema **NP-Hard**. Per trovare una soluzione che si avvicini il più possiible a quella ottimale si usano delle euristiche. Questo algoritmo per trovare una soluzione accettabile è di tipo greedy e non ci garantisce che la soluzione sia ottimale, il punto principale è scegliere l'attributo su cui biforcare in modo da:

- Dividere in insiemi che sono il più possibile puri.
- Porti più rapidamente a dei nodi foglia.

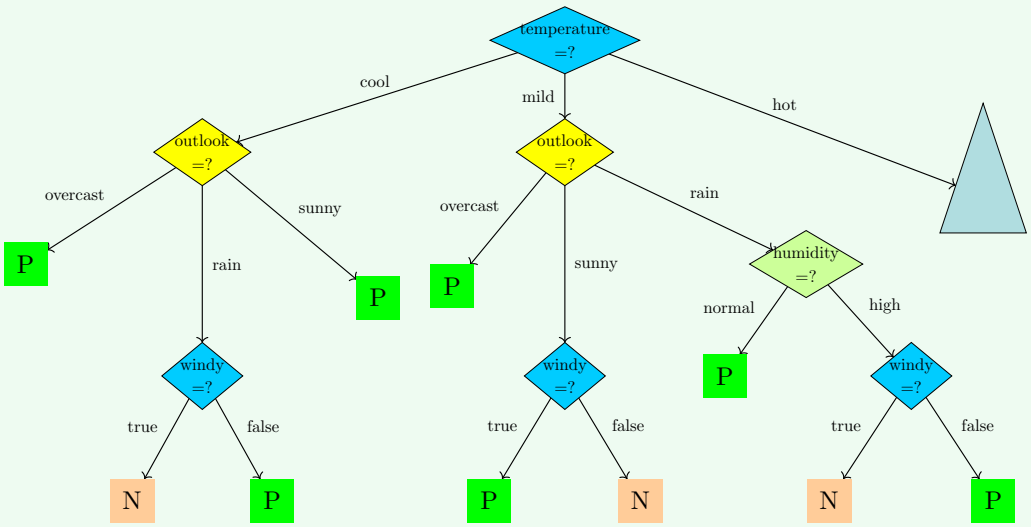
Vogliamo ricondurci ad alberi binari perché sono i più semplici a livello di complessità computazionale quando dobbiamo attraversarli, ed è anche più facile trovare alberi completi.

Guardiamo un esempio di costruzione:

Esempio 2

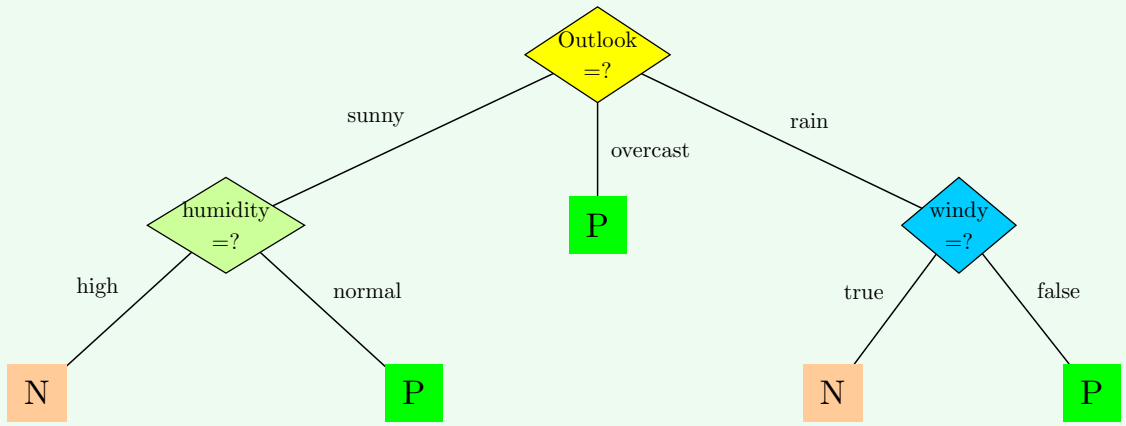
Prendiamo in considerazione una tabella con informazioni sul meteo per stabilire se è possibile giocare:

Outlook	Tempreature	Humidity	Windy	Class
sunny	hot	high	false	N
sunny	hot	high	true	N
overcast	hot	high	false	P
rain	mild	high	false	P
rain	cool	normal	false	P
rain	cool	normal	true	N
overcast	cool	normal	true	P
sunny	mild	high	false	N
sunny	cool	normal	false	P
rain	mild	normal	false	P
sunny	mild	normal	true	P
overcast	mild	high	true	P
overcast	hot	normal	false	P
rain	mild	high	true	N



IF (temperature = cool AND outlook = rain AND windy = false) THEN play

L'esempio qui sopra è un albero decisionale per il problema, ma non è la soluzione migliore. Per esempio possiamo avere un albero equivalente:



6.1.3.2 Goodness function

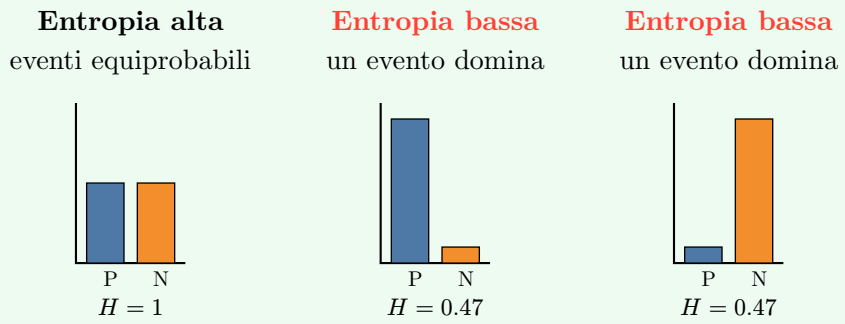
Come funzione di goodness possiamo avere:

- ***Information gain***: lavora su attributi **categoriali**.
- ***Indice di Gini***: lavora su attributi con **valori continui**.

6.1.3.3 Information Gain

Guardiamo più nel dettaglio l'information gain: l'algoritmo ha l'obiettivo di ridurre progressivamente l'entropia. La purezza è definita in base a quanto un insieme di istanze è prossimo alla situazione ideale, ovvero ad una sola etichetta. Avremo quindi come nodi foglia finali insiemi dove l'entropia è bassa.

Esempio 3



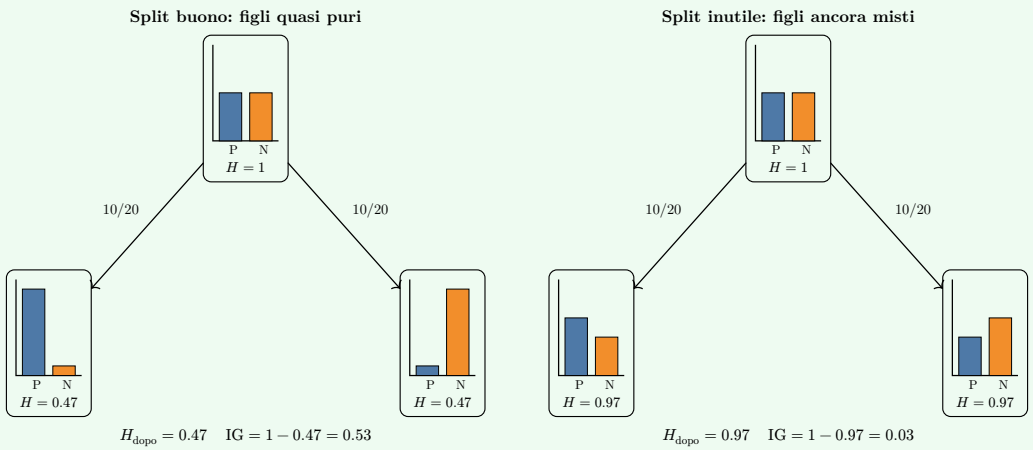


Figura 19. Esempio di calcolo entropia per albero di decisione.

Attenzione

In questo modo stiamo confrontando entropie di sistemi diversi.

Ricordiamo che **NON** si possono confrontare entropie di sistemi diversi. In questo caso è un eccezione, ma **NON** è la norma.

Guardiamo l'esempio qui sopra con 2 classi per la nostra classificazione.

Prendiamo S come un insieme contenente p elementi della classe P ed n elementi della classe N.

Possiamo calcolare la quantità di informazione necessaria per decidere se un esempio arbitrario nell'insieme S stia in P o N è definita come:

$$Info(p,n) = -\left(\frac{p}{p+n}\log_2\frac{p}{p+n}\right) - \left(\frac{n}{p+n}\log_2\frac{n}{p+n}\right)$$

Ipotizziamo ora di utilizzare l'attributo A come radice dell'albero con valori $\{a_1, a_2, ..., a_v\}$, allora S sarà partizionato in $S_1, S_2, ..., S_v$ insiemi. I valori di A e le partizioni di S sono collegate tra loro seguendo l'indice a pedice: a_1 con S_1 e così via.

L'informazione richiesta per l'albero con radice A è ottenuta da una media pesata della quantità di informazione secondo le partizioni che A impone su S:

$$Info_A(S) = \sum_{i=1}^{v} \frac{p_i + n_i}{p+n}\ Info(p_i, n_i)$$

Slide 20 de: «AIALGO_2627_5-classificazione-modelli.pdf» ha la formula con più di due classi.

Se partizioniamo S nei vari S_i a seguito del test A, l'entropia di S sarà la **media ponderata delle entropie** dei singoli sottoinsiemi, ovvero:

$$Info_A(S) = \sum_{i=i}^{n} \frac{|S_i|}{|S|} S_i$$

La quantità di informazione guadagnata dal partizionamento di S secondo l'attributo A sarà:

$$Gain\ (A) = Info\ (S) - Info_A(S)$$

L'attributo A è selezionato se l'information gain è massimo.

Tutto questo funziona se il dataset è bilanciato, nel caso di un dataset sbilanciato (ovvero la maggior parte dei casi reali) l'information gain favorisce i test che hanno molti esiti. La soluzione è **normalizzare** il gain in base alla quantità di informazione di S stesso.

Consideriamo l'informazione contenuta da un messaggio sulla probabile divisione dell'insieme S negli insiemi S_i.

Tale informazione sarà data da:

$$-\log_2 \frac{|S_i|}{|S|}$$

E per analogia con la definizione di $Info(S)$ si ha:

$$SplitInfo(A) = -\sum_{i=1}^{n} \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

Che rappresenta la potenziale informazione generata dalla divisione di S. Da qui, il **gain normalizzato** secondo $SplitInfo(A)$.

$$GainRatio(A) = \frac{Gain\ (A)}{SplitInfo\ (A)}$$

Torniamo all'esempio del meteo:

Esempio 4

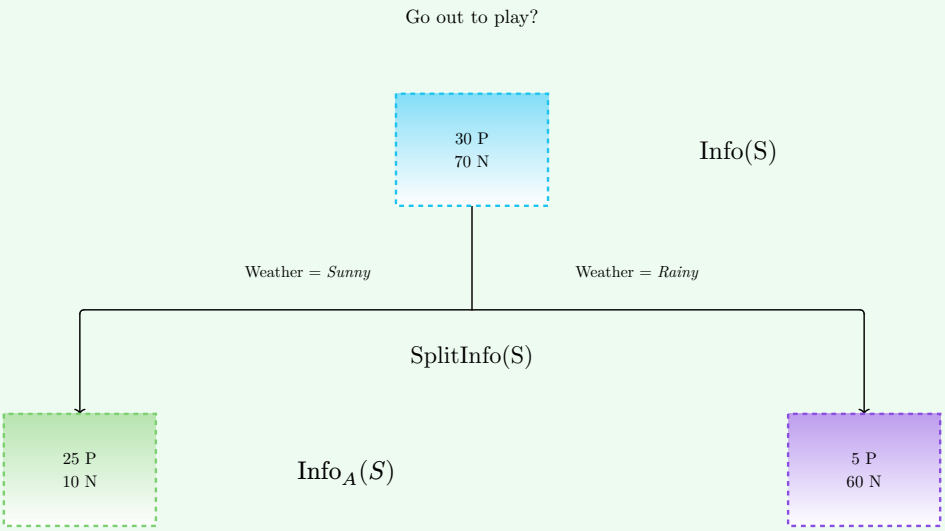


Figura 20. Calcolo di Info(S), SplitInfo(S) e Info$_A(S)$ per lo split su *Weather*.

Purtroppo *GainRatio* non ci permette di risolvere tutti i problemi. Potrebbe capitare che alcuni attributi significativi, che assumono qualche valore in più rispetto agli altri, vengano penalizzati.

Per ovviare si segue questa strategia:

- Si calcola il guadagno per ogni attributo.
- Si calcola la media dei guadagni calcolati.
- Si selezionano **SOLO** gli attributi che hanno guadagno al di sopra della media.
- Si scelgono tra gli attribui selezionati, quelli con *GainRatio* maggiore.

Nota

- Nel caso di attributi di tipo continuo si utilizza un valore soglia Z per stabilire se l'attributo è sopra o sotto al valore di soglia Z.
- Nel caso di attributi con valori mancanti ci sono due approcci:
  - Assegnare il valore più comune al valore nullo.
  - Assegnare la probabilità ad ognuno dei possibili valori.

  Possiamo riscrivere $Gain(A)$ può essere riscritto come:

$$Gain(A) = F \cdot (Info(S) - Info_A(S))$$

  - F, percentuale di casi nel training set per cui A è noto.

  Questo ha effetti anche sul partizionamento, dato che dobbiamo ricalcolare $SplitInfo(S)$ e anche *GainRatio*.

📝 Possibile domanda d'esame

Una possibile domanda aperta all'esame può essere quella di descrivere i metodi di partizionamento.

Esempio 5

Costruiamo l'albero sul dataset del meteo (14 esempi: 9 P e 5 N) con **information gain** e **gain ratio**.

6.1.4 Passo 1: nodo radice

Il nodo contiene 9 P e 5 N, quindi $\text{Info}(S) = -\left(\frac{9}{14}\log_2\frac{9}{14} + \frac{5}{14}\log_2\frac{5}{14}\right) = 0.94$.

Attributo Outlook

Valore	P	N	Righe	*H*
sunny	2	3	5	0.971
overcast	4	0	4	0
rain	3	2	5	0.971

- $\text{Info}_A(S) = \frac{5}{14}\cdot 0.971 + \frac{4}{14}\cdot 0 + \frac{5}{14}\cdot 0.971 = 0.694$
- $\text{Gain} = 0.94 - 0.694 = 0.247$
- $\text{SplitInfo} = -\left(\frac{5}{14}\log_2\frac{5}{14} + \frac{4}{14}\log_2\frac{4}{14} + \frac{5}{14}\log_2\frac{5}{14}\right) = 1.577$
- $\text{GainRatio} = \frac{0.247}{1.577} = 0.156$

Attributo Temperature

Valore	P	N	Righe	*H*
hot	2	2	4	1
mild	4	2	6	0.918
cool	3	1	4	0.811

- $\text{Info}_A(S) = \frac{4}{14}\cdot 1 + \frac{6}{14}\cdot 0.918 + \frac{4}{14}\cdot 0.811 = 0.911$
- $\text{Gain} = 0.94 - 0.911 = 0.029$
- $\text{SplitInfo} = -\left(\frac{4}{14}\log_2\frac{4}{14} + \frac{6}{14}\log_2\frac{6}{14} + \frac{4}{14}\log_2\frac{4}{14}\right) = 1.557$
- $\text{GainRatio} = \frac{0.029}{1.557} = 0.019$

Attributo Humidity

Valore	P	N	Righe	*H*
high	3	4	7	0.985
normal	6	1	7	0.592

- $\text{Info}_A(S) = \frac{7}{14}\cdot 0.985 + \frac{7}{14}\cdot 0.592 = 0.788$
- $\text{Gain} = 0.94 - 0.788 = 0.152$
- $\text{SplitInfo} = -\left(\frac{7}{14}\log_2\frac{7}{14} + \frac{7}{14}\log_2\frac{7}{14}\right) = 1$
- $\text{GainRatio} = \frac{0.152}{1} = 0.152$

Attributo Windy

Valore	P	N	Righe	H
false	6	2	8	0.811
true	3	3	6	1

- $\text{Info}_A(S) = \frac{8}{14} \cdot 0.811 + \frac{6}{14} \cdot 1 = 0.892$
- $\text{Gain} = 0.94 - 0.892 = 0.048$
- $\text{SplitInfo} = -\left(\frac{8}{14}\log_2\frac{8}{14} + \frac{6}{14}\log_2\frac{6}{14}\right) = 0.985$
- $\text{GainRatio} = \frac{0.048}{0.985} = 0.049$

Riepilogo e scelta

Attributo	Gain	SplitInfo	GainRatio	Gain ≥ media?
Outlook	0.247	1.577	0.156	sì
Temperature	0.029	1.557	0.019	no
Humidity	0.152	1	0.152	sì
Windy	0.048	0.985	0.049	no

Media dei gain: $\frac{0.247 + 0.029 + 0.152 + 0.048}{4} = 0.119$. Passano il filtro solo gli attributi con gain ≥ media; tra questi vince il GainRatio maggiore. Si sceglie quindi **Outlook**.

6.1.5 Passo 2: figli di Outlook

Un figlio per ogni valore dell'attributo scelto:

- **overcast**: 4 P e 0 N, $H = 0$. Nodo **puro**: diventa una foglia P.
- **sunny**: 2 P e 3 N, $H = 0.971$. Impuro: si riapplica l'algoritmo.
- **rain**: 3 P e 2 N, $H = 0.971$. Impuro: si riapplica l'algoritmo.

6.1.6 Passo 3: ramo sunny

Outlook è già stato usato: restano Temperature, Humidity e Windy.

Il nodo contiene 2 P e 3 N, quindi $\text{Info}(S) = -\left(\frac{2}{5}\log_2\frac{2}{5} + \frac{3}{5}\log_2\frac{3}{5}\right) = 0.971$.

Attributo Temperature

Valore	P	N	Righe	H
hot	0	2	2	0
mild	1	1	2	1
cool	1	0	1	0

- $\text{Info}_A(S) = \frac{2}{5} \cdot 0 + \frac{2}{5} \cdot 1 + \frac{1}{5} \cdot 0 = 0.4$
- $\text{Gain} = 0.971 - 0.4 = 0.571$
- $\text{SplitInfo} = -\left(\frac{2}{5}\log_2\frac{2}{5} + \frac{2}{5}\log_2\frac{2}{5} + \frac{1}{5}\log_2\frac{1}{5}\right) = 1.522$
- $\text{GainRatio} = \frac{0.571}{1.522} = 0.375$

Attributo Humidity

Valore	P	N	Righe	H
high	0	3	3	0
normal	2	0	2	0

- $\text{Info}_A(S) = \frac{3}{5} \cdot 0 + \frac{2}{5} \cdot 0 = 0$
- $\text{Gain} = 0.971 - 0 = 0.971$
- $\text{SplitInfo} = -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right) = 0.971$
- $\text{GainRatio} = \frac{0.971}{0.971} = 1$

Attributo Windy

Valore	P	N	Righe	H
false	1	2	3	0.918
true	1	1	2	1

- $\text{Info}_A(S) = \frac{3}{5} \cdot 0.918 + \frac{2}{5} \cdot 1 = 0.951$
- $\text{Gain} = 0.971 - 0.951 = 0.02$
- $\text{SplitInfo} = -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right) = 0.971$
- $\text{GainRatio} = \frac{0.02}{0.971} = 0.021$

Riepilogo e scelta

Attributo	Gain	SplitInfo	GainRatio	Gain ≥ media?
Temperature	0.571	1.522	0.375	sì
Humidity	0.971	0.971	1	sì
Windy	0.02	0.971	0.021	no

Media dei gain: $\frac{0.571 + 0.971 + 0.02}{3} = 0.521$. Passano il filtro solo gli attributi con gain ≥ media; tra questi vince il GainRatio maggiore. Si sceglie quindi **Humidity**.

6.1.7 Passo 4: ramo rain

Il nodo contiene 3 P e 2 N, quindi $\text{Info}(S) = -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right) = 0.971$.

Attributo Temperature

Valore	P	N	Righe	H
mild	2	1	3	0.918
cool	1	1	2	1

- $\text{Info}_A(S) = \frac{3}{5} \cdot 0.918 + \frac{2}{5} \cdot 1 = 0.951$
- $\text{Gain} = 0.971 - 0.951 = 0.02$
- $\text{SplitInfo} = -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right) = 0.971$

- GainRatio $= \frac{0.02}{0.971} = 0.021$

Attributo Humidity

Valore	P	N	Righe	H
high	1	1	2	1
normal	2	1	3	0.918

- $\text{Info}_A(S) = \frac{2}{5} \cdot 1 + \frac{3}{5} \cdot 0.918 = 0.951$
- Gain $= 0.971 - 0.951 = 0.02$
- SplitInfo $= -\left(\frac{2}{5}\log_2\frac{2}{5} + \frac{3}{5}\log_2\frac{3}{5}\right) = 0.971$
- GainRatio $= \frac{0.02}{0.971} = 0.021$

Attributo Windy

Valore	P	N	Righe	H
false	3	0	3	0
true	0	2	2	0

- $\text{Info}_A(S) = \frac{3}{5} \cdot 0 + \frac{2}{5} \cdot 0 = 0$
- Gain $= 0.971 - 0 = 0.971$
- SplitInfo $= -\left(\frac{3}{5}\log_2\frac{3}{5} + \frac{2}{5}\log_2\frac{2}{5}\right) = 0.971$
- GainRatio $= \frac{0.971}{0.971} = 1$

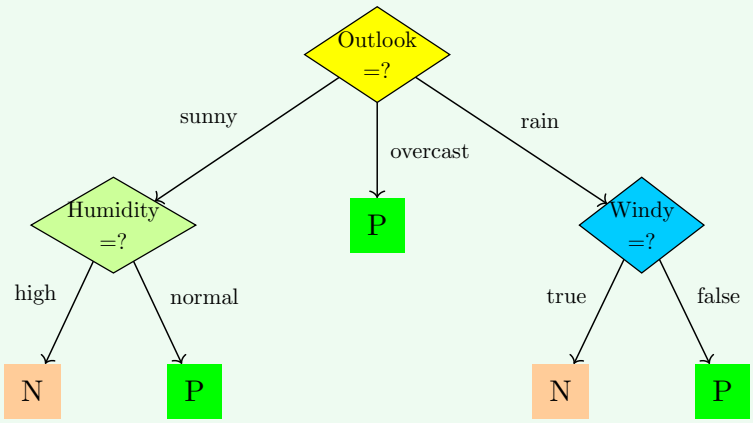
Riepilogo e scelta

Attributo	Gain	SplitInfo	GainRatio	Gain ≥ media?
Temperature	0.02	0.971	0.021	no
Humidity	0.02	0.971	0.021	no
Windy	0.971	0.971	1	sì

Media dei gain: $\frac{0.02 + 0.02 + 0.971}{3} = 0.337$. Passano il filtro solo gli attributi con gain ≥ media; tra questi vince il GainRatio maggiore. Si sceglie quindi **Windy**.

Tutti i nodi sono ora puri: l'algoritmo termina.

6.1.8 Albero finale



7 Lezione 7 del 16 ottobre 2026

La scorsa lezione abbiamo introdotto i metodi di classificazione, lazy o eager, e siamo andati più nel dettaglio per quanto riguarda gli alberi di decisione, modelli di classificazione eager.

Riprendiamo dall'indice di Gini per gli alberi di decisione.

7.1 Indice di Gini

A differenza dell'information gain, che lavora con valori categoriali, l'indice di Gini lavora con valori continui.

Diamo una definizione per l'indice di Gini:

Definizione 49 — Indice di Gini

Dato un insieme di oggetti S contenente n classi, l'indice di Gini è definito come:

$$gini(S) = 1 - \sum_{j-1}^{n} p_j^2$$

dove p_j è la frequenza della classe j-esima di S.

Esempio 6

Date due classi P ed N, tale che $S = P \cup N$ contenenti rispettivamente p ed n elementi, possiamo calcolare:

$$p_P = \frac{p}{p+n}$$

$$p_N = \frac{n}{n+p}$$

$$gini(S) = 1 - p_P^2 - p_N^2$$

Possiamo anche definire l'indice di Gini nel caso in cui S sia suddiviso in due sottoinsiemi S_1 e S_2:

Definizione 50 — Indice di Gini dello split

L'indice di Gini dello split di S possiamo definirlo come:

$$gini_{\text{split}}(S, a) = \frac{N_1}{N}\ gini(S_1) + \frac{N_2}{N}\ gini(S_2)$$

Dove a è l'attributo che divied S in due insiemi.

Generalizzando, se l'attributo a divide S in m sottoinsiemi, ognuno di dimensione N_i, l'indice di impurità dello split è calcolato come:

$$gini_{\text{split}}(S, a) = \sum_{i=1}^{m} \frac{N_i}{N} \, gini(S_i)$$

L'attributo con il minor valore è selezionato coem attributo dello split.

Guardiamo un esempio di indice di Gini:

Esempio 7

Prendiamo in considerazione i seguenti dati:

RID	Age	Car Type	Risk
0	23	family	high
1	17	sport	high
2	43	sport	high
3	68	family	low
4	32	truck	low
5	20	family	high

Attribute list for "Age"

Age	RID	Risk
17	1	high
20	5	high
23	0	high
32	4	low
43	2	high
68	3	low

Attribute list for "Car Type"

Car Type	RID	Risk
family	0	high
sport	1	high
sport	2	high
family	3	low
truck	4	low
family	5	high

Possiamo avere vari possibili valori di split per l'attributo *Age*:

$Age =< 17$, $Age =< 20$, $Age =< 23$, $Age =< 32$, $Age =< 43$, $Age =< 68$.

Iniziamo con il valore 17.

Tuple count	High	Low
Age ≤ 17	1	0

Tuple count	High	Low
Age $>$ 17	3	2

Calcoliamo sia per valori minori o uguali a 17 e maggiori di 17:

$$G(Age =< 17) = 1 - (1^2 + 0^2) = 0$$

$$G(Age > 17) = 1 - \left(\left(\frac{3}{5}\right)^2 + \left(\frac{2}{5}\right)^2\right) = 1 - \frac{13}{25}^2 = \frac{12}{25}$$

Un albero di decisione separa i dati in maniera diversa rispetto a un modello di machine learning. Con un modello di ML possiamo decidere se separare i dati secondo varie curve: retta, parabola o curve più complesse. Un albero di decisione invece cerca di separare i dati con più rette per la classificazione.

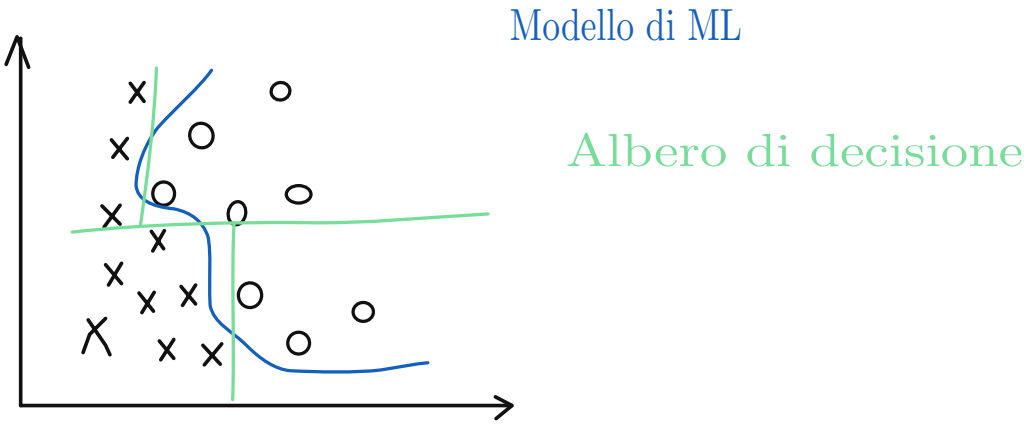


Figura 21. Modello di ML vs Albero di decisione.

7.2 Pruning degli alberi di decisione

Un problema che può sorgere con gli alberi di decisione, è la presenza di un elevato numero di attributi all'interno del training set, questo comporta la crescita dell'albero decisionale.

Dato che un albero di decisione porta ad avere sempre delle foglie pure, gli alberi decisionali tendono ad andare in overfitting sul training set.

Una soluzione è quella di rimuovere rami dall'albero che non portano ad una corretta classificazione, questa tecnica si chiama potatura e abbiamo due modalità:

- **Pre-pruning**: Viene effettuata durante la costruzione dell'albero, si effettua una previsione sulla presunta precisione che il ramo potrebbe avere se decidessimo di costruirlo per intero.
- **Post-pruning**: Viene effettuata una volta che l'albero è stato costruito.

Useremo **post-pruning**, anche se più dispendioso, perché dobbiamo costruire l'albero per intero e poi lavorarci nuovamente sopra, ci permette di essere più precisi rispetto al **pre-pruning**. Per il **post-pruning** l'algoritmo più usato è l'algoritmo C4.5:

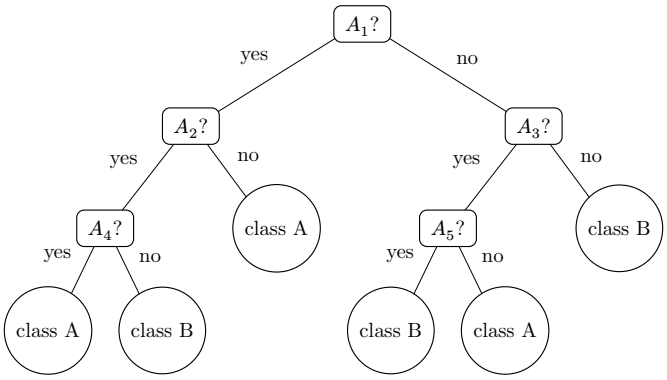


Figura 22. Albero completo

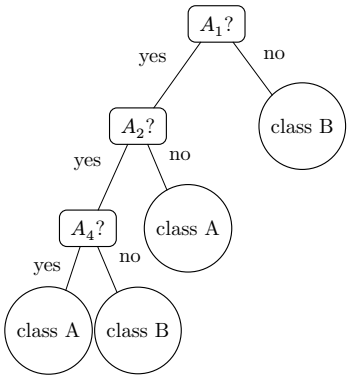


Figura 23. Albero potato con C4.5

Dobbiamo anche stabilere con quale criterio scartare rami dell'albero, altrimenti alcuni casi verranno classificati erroneamente. Una soluzione è quella di specificare una classe di distribuzione, per ogni classe presente nel training set, e la probabilità che un caso appartenga a quella classe. Questo ci permette di, commettere leggermente più errori in generale, ma possiamo generalizzare meglio il nostro modello.

Con l'algoritmo C4.5, quando l'errore medio di un sottoalbero, dopo aver potato uno specifico nodo interno, è minore di quello che riserva il test stesso, il nodo corrispondente al test viene potato insieme a tutto il suo sottoalbero.

7.3 Minimal cost-complexity pruning

Esiste anche l'algoritmo di **minimal cost-complexity pruning** che è un algoritmo di pruning parametrizzato tramite un parametro $\alpha \geq 0$ detto anche **parametro di complessità**.

> **Definizione 51 — Minimal cost-complexity pruning**
>
> Dato un parametro α per definire una misura di costo di complessità $R_\alpha(T)$ per un dato albero T definiamo:
>
> $$R_\alpha(T) = R(T) + \alpha|\tilde{T}|$$
>
> Dove $\alpha|\tilde{T}|$ è il numero di nodi terminali nell'albero T e $R(T)$ è il tasso di errore sui nodi terminali dell'albero T.
>
> L'algoritmo ha come obiettivo trovare un sottoalbero di T che minimizzi $R_\alpha(T)$.

Il costo di un singolo nodo è dato da $R_\alpha(t) = R(t)$. Il ramo T_t è un albero avente t come radice.

In generale l'impurità di un nodo radice è maggiore della somma dell'impurità dei nodi terminali, ovvero $R(T_t) < R(t)$.

Possiamo definire un valore effettivo per α che ci permette di decidere come tagliare l'albero:

$$\alpha_{\text{eff}} = \frac{R(t) - R(T_t)}{|T| - 1}$$

L'algoritmo termina quadno l'α_{eff} minimale del sottoalbero potato è maggiore del valore soglia.

Aumentando il parametro α verrà potata sempre una porzione più grande dell'albero, il che aumenta l'impurità totale delle sue foglie.

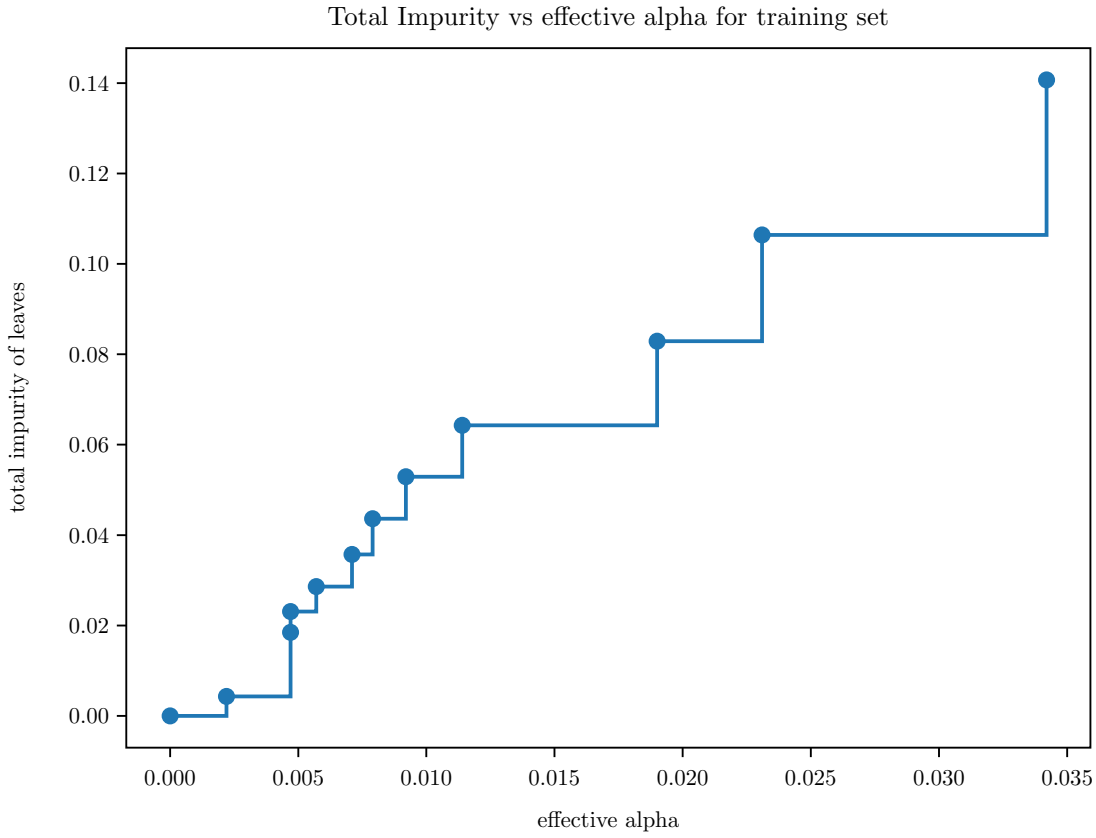


Figura 24. Grafico per alpha effettivo.

Ovviamente, aumentando il valore del parametro α, il numero di nodi e la profondità dell'albero diminuiscono.

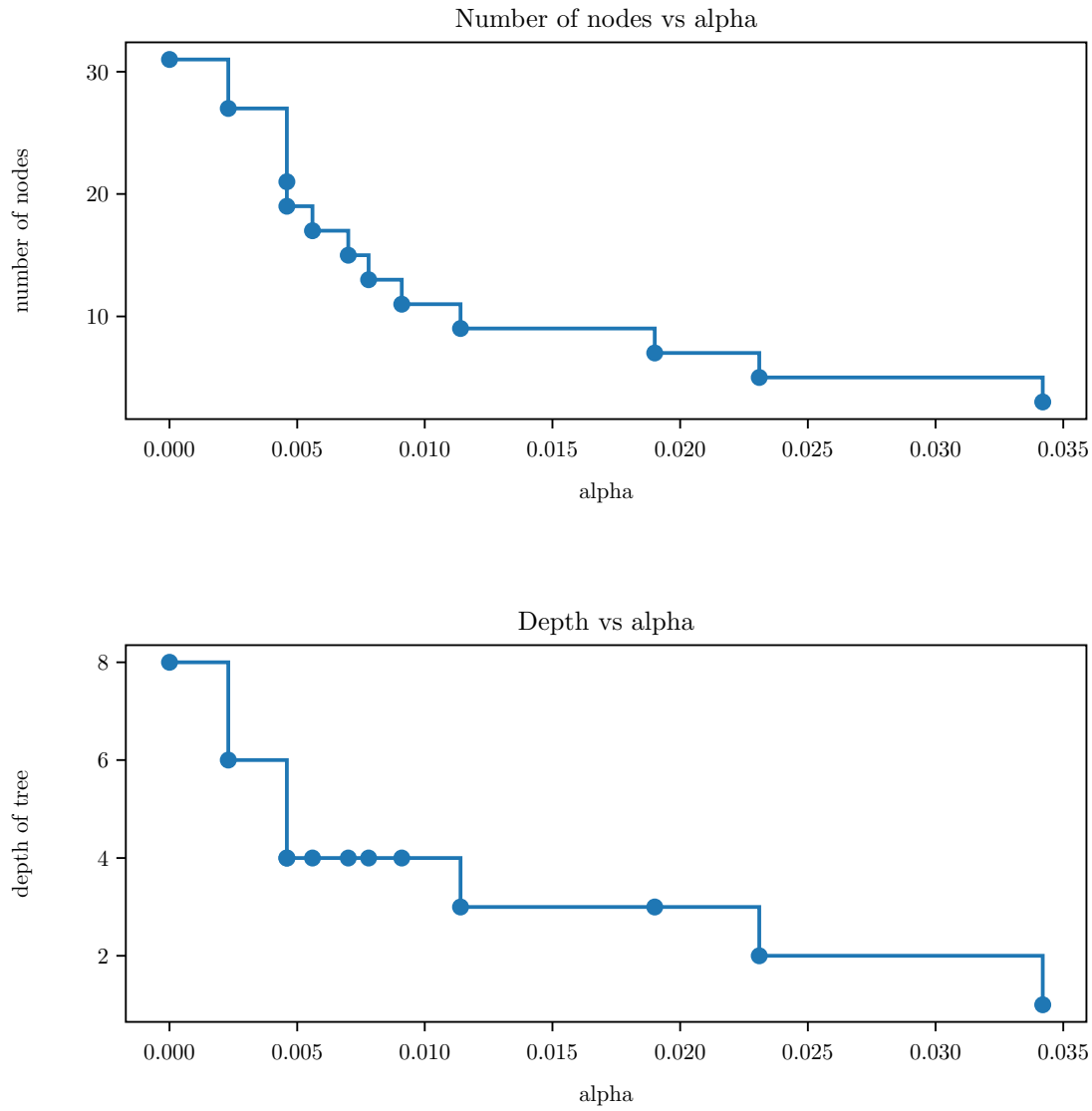


Figura 25. Andamento dei nodi e della profondità dell'albero all'aumentare di alpha.

Un altro aspetto importante è come incide il parametro α sulle prestazioni del modello. Valori bassi tenderanno a mandare il classificatore in overfitting, peggiorando le prestazioni sul test set. Mentre valori più alti tenderanno ad abbassare la precisione durante la fase di training, ma aumentandola nella fase di test. Attenzione che dopo una certa soglia α il modello perde prestazioni sia in training che test set.

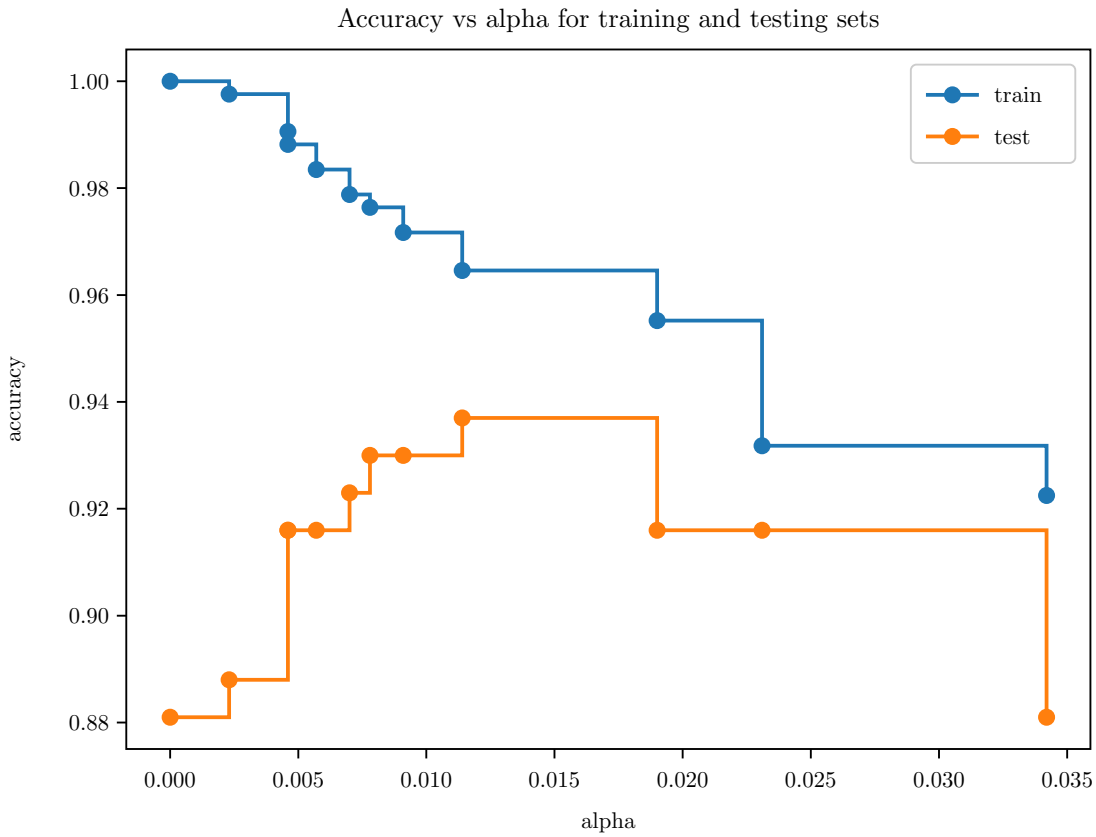


Figura 26. Accuratezza del modello al variare di alpha.

> **Nota**
>
> Se nel proprio progetto per *Laboratorio di Algoritmi per l'intelligenza artificiale* si decide di usare un albero di decisione, è obbligatorio fare il grafico riportato sopra.

Gli alberi di decisione sono modelli semplici da realizzare, hanno anche un'ottima efficienza computazionale e i risultati sono interpretabili. Non sono esenti da difetti, non sono stabili; dei cambiamenti nel training set comportano cambiamenti anche nella struttura dell'albero, di conseguenza i risultati che si ottengono hanno inoltre un'elevata varianza.

7.4 Metodi di consenso

Per risolvere le problematiche degli alberi di decisione ci sono diverse strategie basate sui metodi di consenso. Abbiamo **bagging**, **random forests** e **boosting**.

Queste tecniche ci permettono di guadagnare in accuratezza ma con lo svantaggio di perdere parzialmente l'interpretabilità del modello.

I metodi di consenso impiegano una combinazione di modelli M^*, ottenuta come combinazione di k modelli addestrati separatamente per incrementare l'accuratezza.

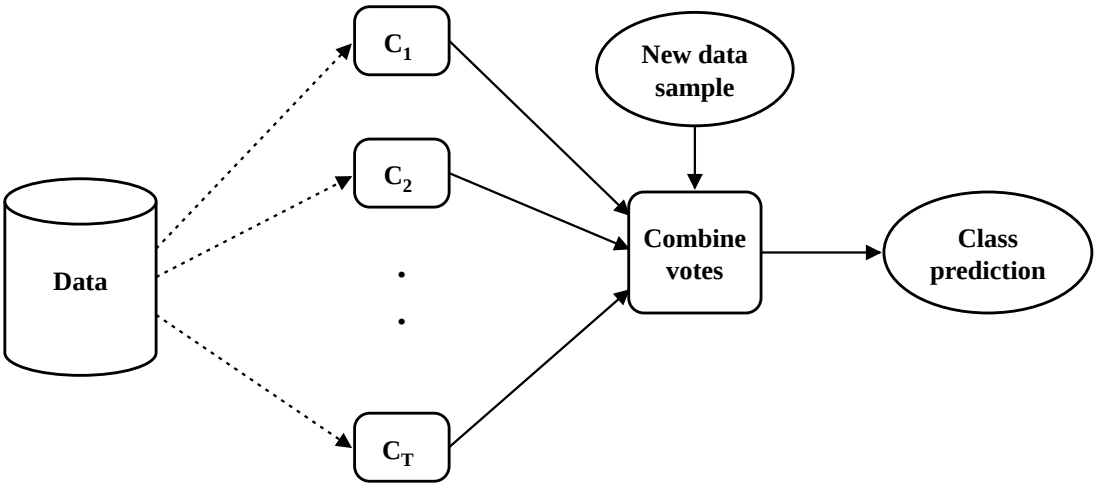


Figura 27. Struttura metodi di consenso.

I metodo più usati sono:

- **Bagging**: si media la predizione su un insieme omogeneo di classificatori.
- **Boosting**: si impiega una collezione di classificatori tramite un **voto pesato**.
- **Ensemble**: si combina un insieme eterogeneo di classificatori.

7.4.1 Bagging

Il **boostrap aggregation**, o **bagging**, è una tecnica utilizzata prevelatemente per ridurre la varianza di un metodo di apprendimento statistico o per stimare l'accuratezza di un modello di machine learning.

La tecnica di **bagging** è particolarmente utile per gli alberi di decisione.

L'idea del boostrap è di estrarre casualmente B diversi sottoinsiemi di osservazioni dall'unico training set per simulare la variabilità dei dati. Addestrare un classificatore f_i per ogni sottoinsieme i separatamente e infine aggregare le predizioni sulle nuove istanze x del test:

$$f_{\text{bag}}(x) = \frac{1}{B}\sum_{i=1}^{B} f_i(x)$$

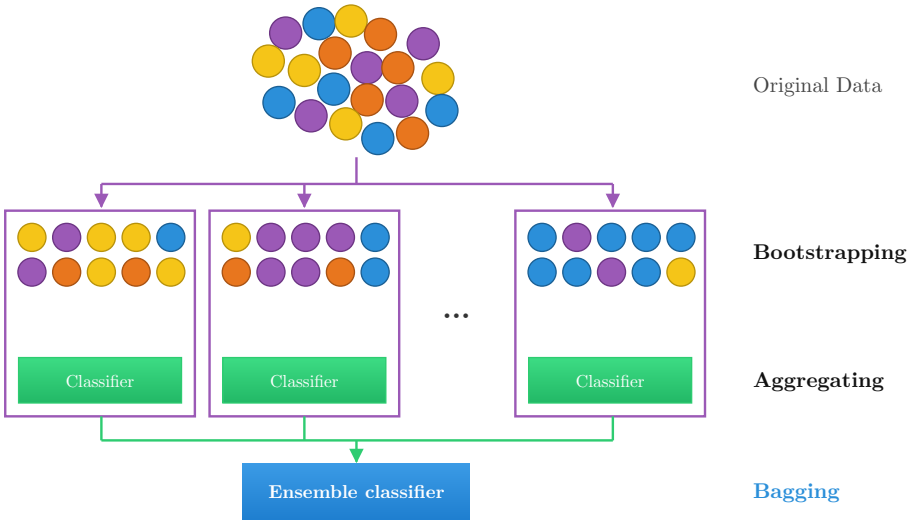


Figura 28. Schema della tecnica di bagging.

7.4.2 Random forest

È un metodo di consenso costituito da un bagging di **alberi di decisione non potati** e complessi, con una scelta casuale di un sottoinsieme delle feature ad ogni split. È un metodo non lineare, ha una bassa varianza grazie all'elevata quantità di alberi dentro la foresta.

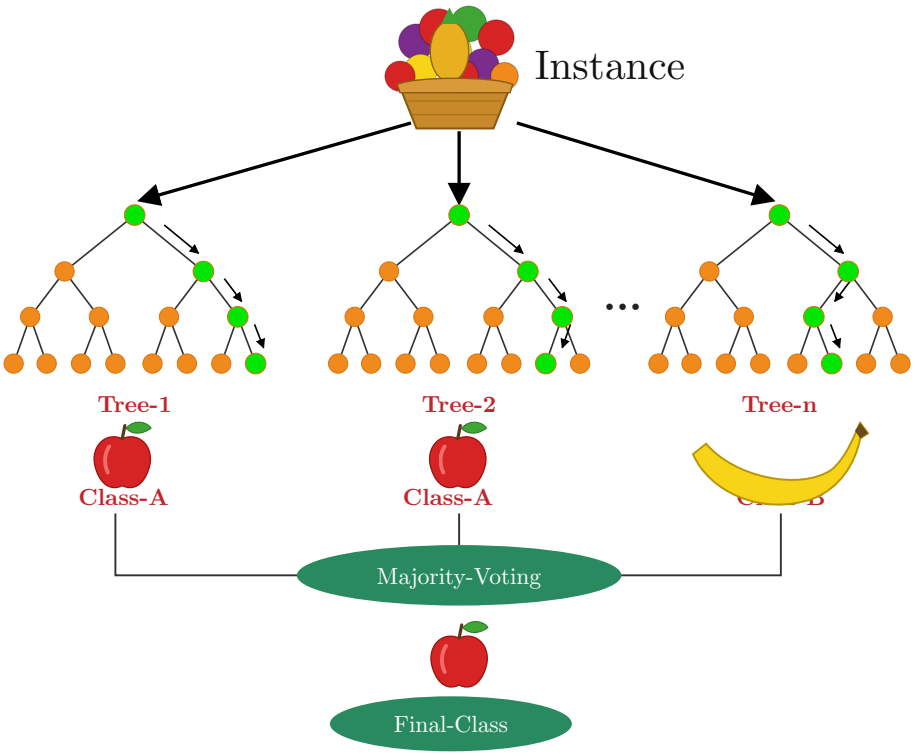


Figura 29. Metodo random forest.

Il metodo random forest è utile anche per feature selection. Gli attributi che sono più significativi per la classificazione tendono a comparire più vicini alla radice degli alberi, questo perché i primi split dalla radice permettono di ottenere maggiori guadagni secondo l'indice di Gini o l'information gain.

Il metodo random forest perde facilmente interpretabilità, dovuta alla presenza di molti alberi di decisione, ma ci permette di **scalare** molto bene permettendo l'esecuzione in parallelo di tutti gli alberi della foresta.

7.4.3 Addestramento della foresta

Fissiamo un numero a piacere n_{tree} e il numero delle feature m_{try} da scegliere casualmente tra quelle disponibili. Di solito m_{try} è posto a $\frac{m}{3}$ per problemi di regressione, e $\sqrt{m}$ per problemi di classificazione, dove m rappresenta il numero totale delle feature.

Per n_{tree} volte ripetiamo la procedura:

Definizione 52 — Algoritmo random forest

1. Scegliere un sottoinsieme di boostrap S dai dati di training.

2. Addestrare un albero di decisione su S con $minsplit = 1$, in cui ad ogni split solo m_{try} preditorri sono scelti casualmente coem candidati per lo split tra tutte le feature.
3. Non potare l'albero.
4. Salvare l'albero ottenuto.

Per la predizione di nuove istanze si aggregano le predizioni di tutti gli alberi, come nel bagging, e si stabilisce la classificazione di una nuova istanza in base alla maggioranza delle predizione fatte dagli n_{tree} (classificazione) alberi o mediante una media delle stesse (regressione).

Random forest non presenta problemi di **overfitting**, dovuto al fatto che facciamo bagging, all'aumentare del numero di alberi. Gli alberi addestrati dentro random forest sono indipendenti, rendendolo parallelizzabile come già detto precedentemente.

L'idea del bagging è quella di calcolare la media di molti modelli contenenti errore ma approssimativamente non distorti, in modo da ridurre la varianza. Gli alberi sono perfetti per il bagging, perché permettono di catturare strutture di interazioni complesse all'interno dei dati, e se fatti crescere con abbastanza profondità, presentano una distorsione relativamente bassa.

Gli alberi di decisione hanno un'elevata varianza, e il bagging la riduce in modo efficace: mediando le previsioni di molti alberi della foresta, gli errori commessi dai singoli alberi tendono a compensarsi.

È importante anche stabilire il numero di alberi all'interno della random forest, un numero basso di alberi aumenterà il livello di errore della RF, mentre un numero più alto tenderà ad abbassarlo come già visto in precedenza. Generalmente si usa un numero intorno al 100 per n_{tree}.

Nota

Se si usa il metodo random forest per il progetto di laboratorio questo grafico qui sotto è essenziale.

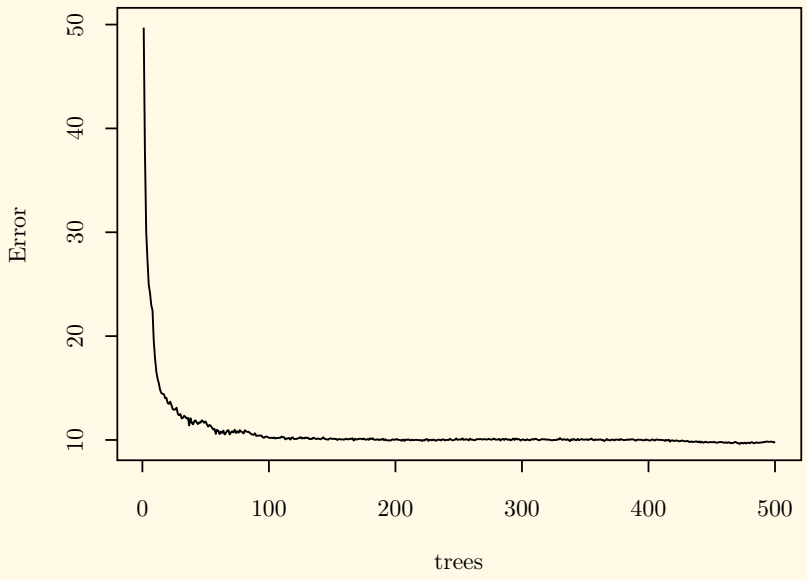


Figura 30. Random forest con diversi n_{tree}.

Le random forest forniscono anche una stima dell'errore di test, chiamato **Out Of Bag (OOB) error**, che è la media degli errori di predizione su ogni istanza di training x_i mediante l'aggregazione delle predizioni dei soli alberi che non contenevano x_i tra le istanze di training.

Per calcolare l'importanza di goni feature si possono applicare le seguenti strategie:

- **Mean decrease in accuracy**: Viene calcolata l'accuratezza della RF sui dati out of bag, quindi per ogni variabile si permuta una colonna alla volta dei dati in modo da individuare quali attributi/feature siano rilevanti per la classificazione. Si calcola la differenza dell'accuratezza per ogni albero sui dati OOB prima e dopo la permutazione per stabilire se una feature è rilevante oppure no; si calcola poi la media sulla differenza calcolata per ogni albero della RF.
- **Mean decrease in the Gini index**: per ogni albero e per ogni variabile i, si considerano tutti i nodi in cui lo split è fatto sulla variabile i. Per ciascuno si calcola la diminuzione dell'indice di Gini (impurità), cioè la differenza tra l'impurità del nodo padre e la media pesata delle impurità dei nodi figli, pesandola per il numero di istanze che arrivano nel nodo. Queste diminuzioni si sommano su tutti i nodi dell'albero che usano la variabile i, e si fa poi la media su tutti gli alberi della RF. Più la diminuzione media è grande, più la variabile è importante.

7.4.4 Riepilogo random forest

Non sempre conviene utilizzare le random forest. In problemi di regressione possono essere poco adatte, perché le previsioni sono medie dei valori osservati nel training: il modello non è in

grado di estrapolare oltre l'intervallo dei dati e produce previsioni a gradini. Nei problemi di classificazione questo limite non c'è.

Inoltre, quando i dati sono scarsi, la random forest ha poco senso: il bootstrap e la selezione casuale delle variabili lavorano su pochi campioni, e gli alberi risultano troppo instabili o troppo simili tra loro perché la media riduca davvero la varianza.

Vantaggi:

- Può eseguire sia attività di regressione che classificazione.
- Una foresta casuale produce buone predizioni che possono essere comprese facilmente.
- Può gestire in modo efficiente set di dati di grandi dimensioni.
- L'algoritmo della foresta casuale fornisce un livello di precisione più elevato nella previsione dei risultati rispetto all'algoritmo dell'albero decisionale.

Svantaggi:

- Quando si usa una random forest, sono necessarie più risorse per il calcolo.
- Consuma più tempo rispetto ad un algoritmo di albero decisionale.

7.5 Boosting

Il boosting prevede l'utilizzo di un insieme di classificatori con voto pesato.

Per esempio è come consultare diversi medici per una diagnosi sulla base di una combinazione di diagnosi pesate, il peso viene assegnato in base all'accuratezza della diagnosi precedente.

Procedimento:

1. Vengono assegnati dei pesi ad ogni tupla del training set.
2. Si addestra una serie di k classificatori
   - Ogni volta che l'i-esimo classificatore M_i viene addestrato, i pesi sono aggiornati in modo da consentire al successivo classificatore, M_{i+1} di porre più attenzione alle tuple che sono state classificate erroneamente dal classificatore M_i.
3. Il classificatore finale M^* combina i voti di ogni classificatore intermedio, tale che il peso assegnato ad ogni voto è proporzionale all'accuratezza dello specifico classificatore.

7.5.1 Boosting: Adaboost

Definizione 53 — Adaboost

Prendiamo in considerazione un insieme D di d tuple etichettate, $(X_1, y_1), ..., (X, y_d)$, dove X_i è l'i-esima tupla e y_i è l'etichetta ad essa assegnata.

Settiamo inizialmente tutti i pesi allo stesso valore di $\frac{1}{d}$.

Generiamo i k classificatori in k passi. Ad ogni passo i:

1. Le tuple nell'insieme D vengono campionate con replacement, ovvero rimettendone uan copia in D dopo l'estrazione, per formare un training set D_i della stessa dimensione di D.
2. Le probabilità di ogni tupla di essere selezionata è proporzionale al suo peso.
3. Viene addestrato un classificatore M_i da D_i.

4. Viene calcolato il tasso di errore M_i utilizzando D_i come test set.
5. Se una tupla viene classificata erroneamente, il suo peso viene incrementato, altrimenti viene decrementato.

Definizione 54 — Adaboost, calcolo errore

Sia $err(X_j)$ l'errore nel classificatore la tupla X_j ($\{0, 1\}$ se si parla di classificatori e non di regressori).

L'errore del classificatore M_i è dato da:

$$error(M_i) = \sum_{j=1}^{d} w_j \times err(X_j)$$

- w_j, peso corrente per la tupla X_j.

Il peso del classificatore M_i nella votazione consenso sarà:

$$\log\left(\frac{1 - error(M_i)}{error(M_i)}\right)$$

8 Lezione 8 del 19 ottobre 2026

Abbiamo visto random forest nella scorsa lezione e i metodi di consenso. Adesso vediamo i Support Vector Machine come metodo di classificazione o regressione

8.1 Support Vector Machine

È uno degli strumenti più utilizzati per la classificazione di pattern in machine learning.

L'obiettivo è quello di determinare le superfici decisionali tra le classi (**classification boundaries**).

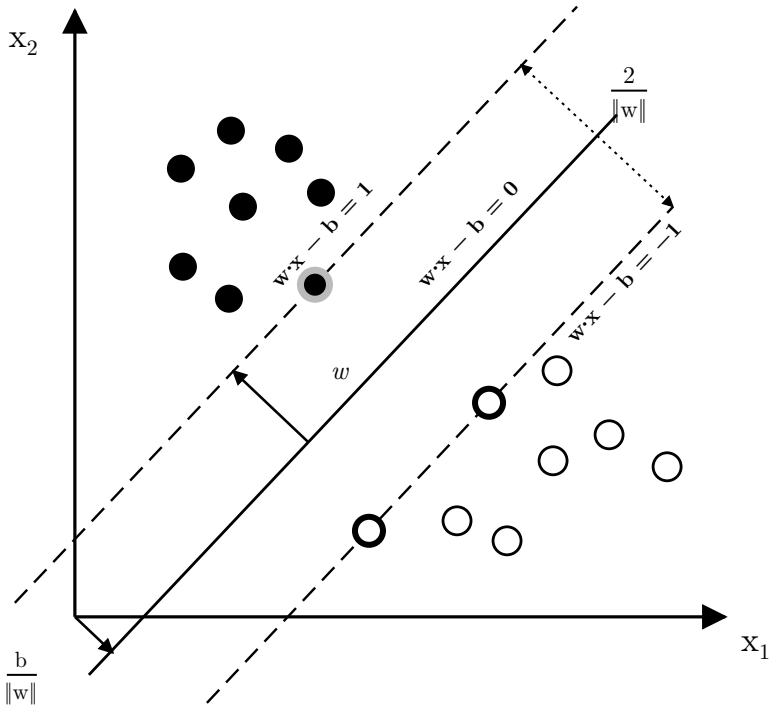


Figura 31. Esempio Support Vector Machine.

La tecnica SVM nasce come classificatore binario, estendibile a più cassi:

- SVM **lineare**: a superficie di separazione è un **iperpiano**, tra i pattern del training set linearmente separabili.
- SVM **lineare** e patter **non linearmente separabili**: Ci saranno inevitabilmente errori di classificazione nel training set dato che non esiste un iperpiano che separi i pattern.
- SVM **non lineare**: superficie di separazione complessa senza ipotesi di separabilità dei pattern.
- Estensione per casi **multiclasse**.

Per un problema di classificazione possiamo avere più iperpiani possibili per poter dividere le due classi, teniamo in considerazione il caso con due classi che è quello più semplice. SVM determina l'iperpiano in grado di separare le classi con il maggior **margine** possibile.

Il margine è la distanza minima dei punti delle due classi ne training set dell'iperpiano individuato.

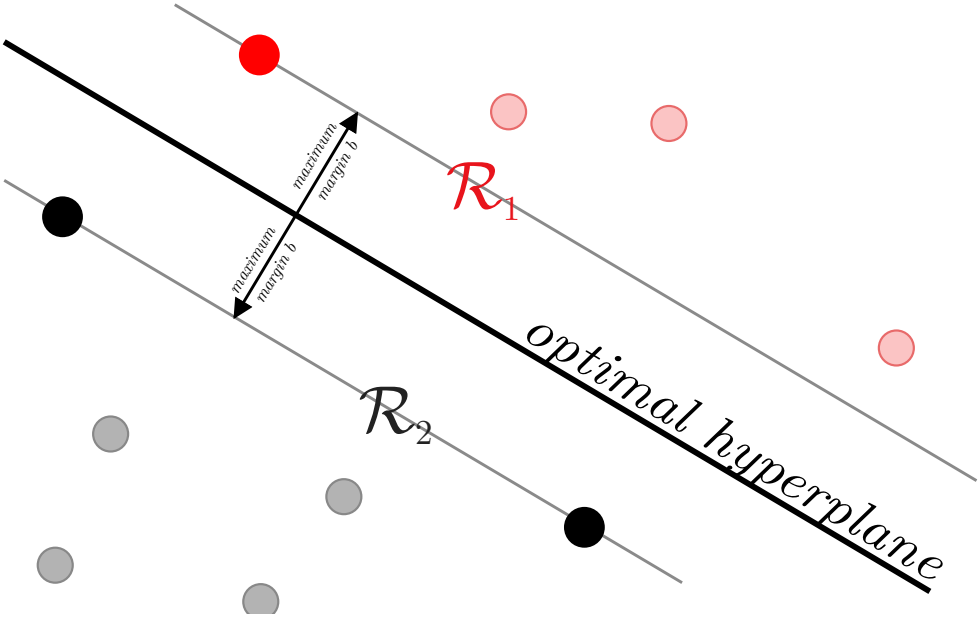


Figura 32. Margine e iperpiano ottimale.

Avere un ampio margine permette una miglior generalizzazione del modello. Con un ampio margine di classificazione dei record del training set, ci permette di poter generalizzare meglio il processo di classificazione per eventuali dati che risiedono in prossimità del confine tra le due classi. Il problema può essere impostato come un'ottimizzazione convessa (convex optimization), questo ammette un solo minimo gobale.

8.1.1 SVM Lineari

Date due classi linearmente separabili, e un training set TS contenente n campioni di coordinate $(x_1, y_1), ...(x_n, y_n)$ dove $x_i \in \mathbb{R}^d$, sono i record multidimensionali e $y_j \in \{+1, -1\}$ sono le etichette delle due classi.

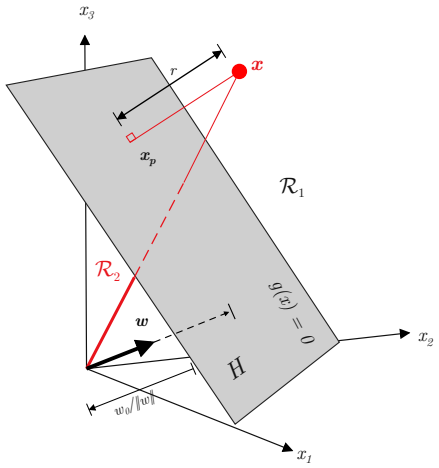


Figura 33. Iperpiano in 3D.

Definizione 55 — Iperpiano

Un generico iperpiano è definito dai parametri (w, b):

$$D(x) = w \cdot x + b$$

- w, vettore normale all'iperpiano.
- $\frac{b}{\|w\|}$, è la distanza dall'origine.
- $D(x) = 0$, è il luogo dei vettori sul piano.

Gli iperpiani (w, b) che separano i dati del TS, con distanza minima $\frac{1}{\|w\|}$ su ogni lato, soddisfano per $i = 1, ..., n$ le seguenti equazioni:

$$w \cdot x_i + b \geq +1 \quad \text{se } y_i = +1$$

$$w \cdot x_i + b \leq -1 \quad \text{se } y_i = -1$$

O in modo più compatto:

$$y_i[w \cdot x_i + b] \geq 1 \quad \text{per } i = 1, ..., n$$

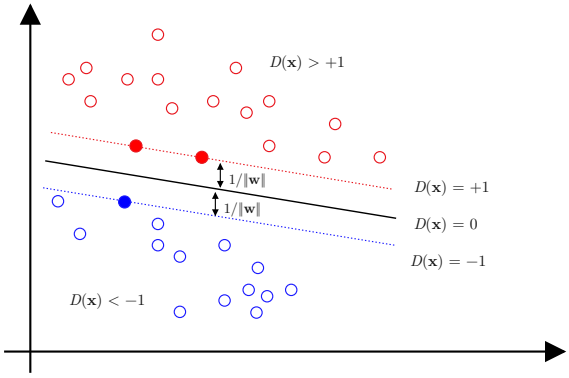


Figura 34. Esempio SVM.

La minima distanza tra l'iperpiano di separazione e un dato di training set è detto **margine** (τ).

La distanza dei punti che giacciono sull'iperpiano $D(x) = +1$ dall'iperpiano di separazione $D(x) = 0$ è $\frac{1}{\|w\|}$; lo stesso vale per i punti sull'iperpiano $D(x) = -1$. Pertanto il margine è $\tau = \frac{2}{\|w\|}$.

8.1.2 Iperpiano ottimo

L'iperpiano ottimo secondo SVM è quello che soddisfa i vincoli di separazione dei dati e massimizza i margine τ.

- Minimizzare: $\frac{\|w\|^2}{2}$
- Vincoli: $y_i[w \cdot x_i + b] \geq 1 \quad \text{per } i = 1, ..., n$

I dati del training set che giacciono sul margine, i cerchi pieni in figura Figura 34, sono detti **support vector**. Tali dati, che costituiscono i casi più complessi, definiscono completamente la soluzione del problema, che può essere espressa come funzione di solo tali dati indipendentemente dalla dimensionalità dello spazio d e dal numero n di elementi in TS.

Uno dei vantaggi di SVM è che permette di scalare molto bene rispetto alla dimensionalità d delo spazio delle feature.

8.2 SVM per dati non separabili

Nel caso in cui avessimo dati non separabili? In questo caso dobbiamo rilassare i vincoli di separazione per far si che alcuni pattern possano valicare il confine della classe. L'obiettivo è quello di ridurre al minimo il numero di questi pattern fuori dalla propria classe d'appartenenza.

A tal fine si introducono n variabili di slack positive ξ_i per $i = 1, ..., n$ e si modificano i vincoli di seprazione in: $y_i[w \cdot x_i + b] \geq 1 - \xi_i \quad \text{per } i = 1, ..., n$

Per ogni dato x_i del TS, la variabile ξ_i codifica la deviazione dal margine. Per i dati separabili del TS le corrispondenti variabili di slack assumono valore 0.

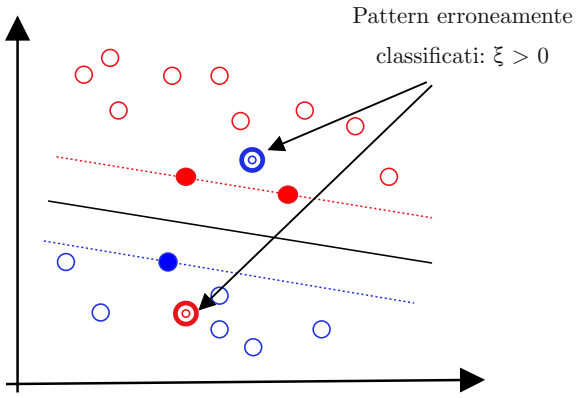


Figura 35. SVM per dati non separabili.

In questo caso, l'iperpiano ottimale deve ancora **massimizzare il margine**, ma allo stesso tempo **minimizzare il numero di elementi non correttamente classificati**. Di conseguenza il nuovo problema di ottimizzazione diventa:

- **Minimizzare**: $\frac{\|w\|^2}{2} + C \sum_{i=1,...,n} \xi_i$
- **Vincoli**: $y_i[w \cdot x_i + b] \geq 1 \quad \text{per } i = 1, ..., n$

Il coefficiente C del problema di ottimizzazione indica l'importanza relativa degli errori di classificazione rispetto all'ampiezza del margine.

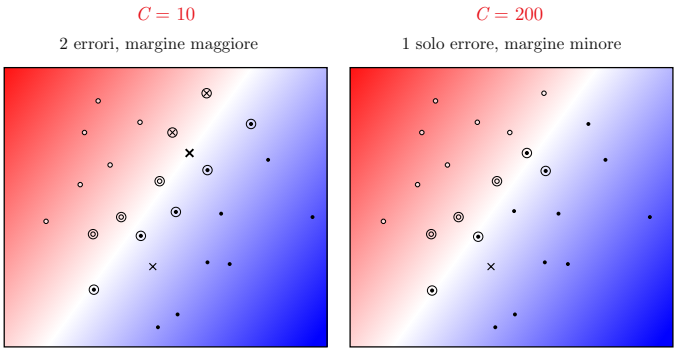


Figura 36. Confronto valori di C.

8.3 SVM non lineari

Una tecnica che può tornare utile è quella di passare ad uno spazio dimensionale più grande di quello di partenza. Questo magari se nel nostro spazio di partenza non siamo in grado separare le classi, con l'aggiunta di una nuova o più dimensioni possiamo trovare un iperpiano che separi le due classi.

Definiamo quindi un **mapping non lineare** dei dati dello spazio di partenza $\mathbb{R}^d$ verso uno spazio $\mathbb{R}^m$ a più dimensionalità, $m > d$.

$$\Phi : R^d \to R^m, \quad \Phi(x) = [g_1(x), g_2(x), ...g_m(x)]$$

Nello spazio $\mathbb{R}^m$, dove ci permette di avere più gradi di libertà, i dati $\Phi(x_1), \Phi(x_2), ..., \Phi(x_n)$ possono essere più facilmente separati da un iperpiano.

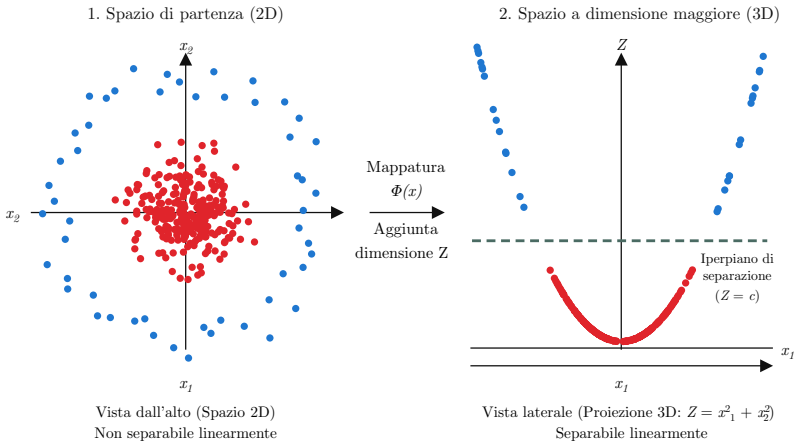


Figura 37. Separazione classi in una dimensione maggiore di quella partenza.

8.4 SVM: Estensione multiclasse

Come possiamo separare più di due classi nello stesso piano dimensionale? Gli approcci più utiizzati al momento sono:

8.4.1 One against one

Troviamo gli iperpiani di separazione a coppie di classi:

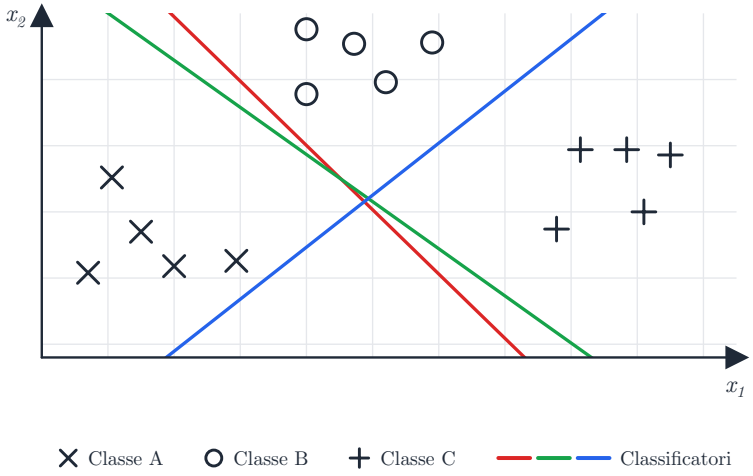


Figura 38. Tecnica SVM: One Against One.

8.4.2 One against all

La one against all funziona nel seguente modo:

- Date s classi $w_1, w_2, ..., w_s$
- Per ogni casse w_k si determina con SVM la superficie di separazione tra i dati di w_k (etichetta $+1$) da una parte, e i dati di tutte le rimanenti classi con $w_{h \neq k}$ (etichettati con -1) dall'altra. Si ottiene la funzione $D_k(x)$ che indica quanto x è distante dalla superficie decisionale in direzione w_k. Maggiore è $D_k(x)$ più confidenti siamo dell'appartenenza di x a w_k.
- Al termine del training si assegna il dato X ala classe k^* per cui è massima la distanza dalla superficie decisionale.

Non è necessario eseguire s training SVM diversi.

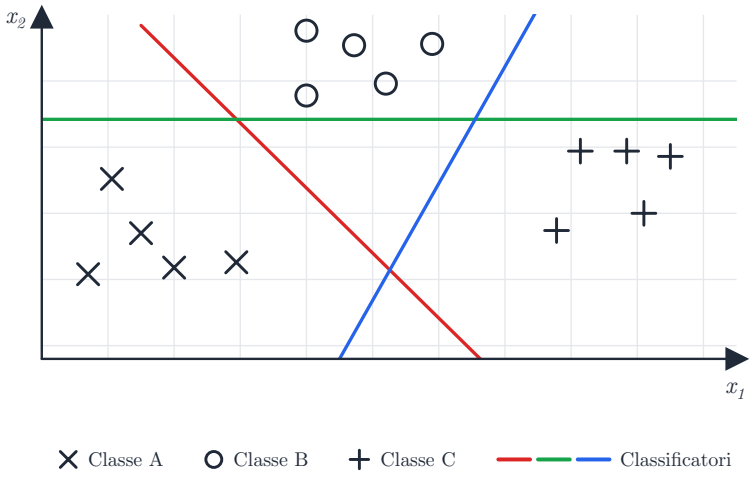


Figura 39. Tecnica SVM: One against all.

In entrambe le tecniche si stabilisce dove cadano nuovi dati da classificare in base ai quadranti che nascono dagli iperpiani.